

Uniwersytet Rzeszowski

Wydział Nauk Ścisłych i Technicznych



Jakub Płocica 134960

PayItOff – System do monitorowania, zarządzania i rozliczania finansów grupowych w architekturze rozproszonej

Dokumentacja techniczna

Projekt zaliczeniowy przedmiotu Programowanie Obiektowe 2

Prowadzący:

mgr inż. Wojciech Gałka

Rzeszów, 2026 r.

Spis treści

1	Wstęp	3
1.1	Cel projektu	3
1.2	Funkcjonalności systemu	3
2	Architektura Aplikacji	4
2.1	Warstwy i technologie	4
2.2	Wzorzec MVVM w aplikacji klienckiej	4
3	Specyfikacja REST API	5
4	Opis Bazy Danych	8
4.1	Schemat bazy (Diagram ERD)	8
4.2	Opis poszczególnych tabel	9
5	Wygląd i przepływ w aplikacji (.NET MAUI)	13
5.1	Krok 1: Inicjacja (Rejestracja i Logowanie)	13
5.2	Krok 2: Konfiguracja, Nawigacja i Zarządzanie Kontem	15
5.3	Krok 3: Budowanie Sieci Kontaktów (Friends)	21
5.4	Krok 4: Tworzenie Ekosystemu (Grupy Wydatków)	25
5.5	Krok 5: Księgowanie Rachunków (Wydatki)	31
5.6	Krok 6: Inspekcja i Monitorowanie	38
5.7	Krok 7: Zamykanie Cyklu (Rozliczanie Długów)	41
5.8	Krok 8: Centrum Powiadomień	44
5.9	Krok 9: Archiwizacja i Retencja Danych	47
6	Harmonogramowanie Zadań w Tle (Hangfire)	50
7	Opis Wybranych Fragmentów Kodu Źródłowego	51
7.1	Algorytm dzielenia rachunku (Penny-Drop Problem)	51
7.2	Logowanie i zapisywanie tokenu JWT	51
7.3	Globalna Obsługa Błędów i Walidacja potoku HTTP	52
7.4	Odpytywanie bazy przy użyciu Entity Framework	53
7.5	Tworzenie wydatków z użyciem Transakcji Bazy Danych	53
7.6	Architektura MVVM i powiązanie z interfejsem	54
7.7	Ewaluator Wyrażeń Matematycznych (Math Parser) w polu ceny	55
7.8	Reaktywny algorytm podziału kosztów na żywo (Client-Side)	55
7.9	Automatyczna Iniekcja Tokenu JWT w aplikacji mobilnej	56
7.10	Powiadomienia E-mail przy użyciu MailKit i protokołu SMTP	57

7.11 Dostosowanie platformy Windows (MAUI Desktop Handlers)	57
8 Instrukcja Uruchomienia Aplikacji	59
8.1 Wymagania wstępne	59
8.2 Krok 1: Uruchomienie Serwera i Bazy Danych (Backend)	59
8.3 Krok 2: Uruchomienie Klienta Desktopowego (.NET MAUI)	59
9 Repozytorium GitHub	61
10 Podsumowanie i Wnioski	62

1. Wstęp

1.1. Cel projektu

Aplikacja **PayItOff** to projekt stworzony w celu łatwego zarządzania wspólnymi kosztami i długami ze znajomymi. Głównym założeniem było napisanie narzędzia, które uwolni użytkowników od problemu ręcznego obliczania, kto jest komu winien pieniądze po wspólnych wakacjach czy wyjściu na miasto. System automatycznie przelicza podziały rachunków i zarządza grupami użytkowników.

1.2. Funkcjonalności systemu

- **Zarządzanie profilem:** Możliwość rejestracji, bezpiecznego logowania (Tokeny JWT) oraz edycji danych użytkownika (zdjęcie profilowe, hasło).
- **Grupy wydatków:** Moduł pozwalający na tworzenie grup tematycznych (np. Mieszkanie, Wyjazd), do których przypisywani są znajomi.
- **Podział paragonów:** Dodawanie kosztów i możliwość precyzyjnego dzielenia ich na poszczególne osoby – w równych częściach lub w konkretnych kwotach/procentach.
- **Lista długów i spłaty:** Czytelny wykaz pokazujący, ile zalegamy innym użytkownikom. System posiada opcję odnotowania częściowej lub pełnej spłaty (przelewu).
- **Lista znajomych i powiadomienia:** Wbudowany system zaproszeń do kontaktów i wyświetlanie alertów, gdy ktoś doda nowy wydatek.

2. Architektura Aplikacji

2.1. Warstwy i technologie

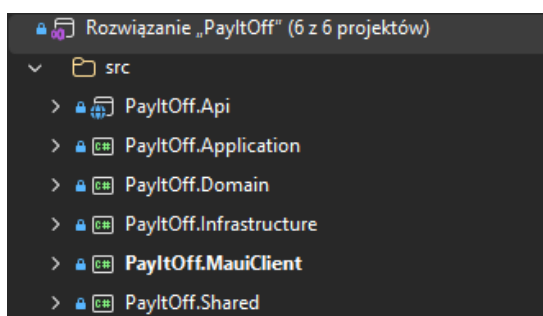
Zdecydowałem się podzielić aplikację na dwie główne części: warstwę serwera (backend) i aplikację użytkownika (frontend). Ułatwia to zarządzanie kodem i bezpieczeństwo bazy danych.

- **Backend (API):** Napisany w **ASP.NET Core 10 Web API**. Komunikacja z bazą danych opiera się na bibliotece **Entity Framework Core**. Zabezpieczenia i logowanie zrealizowano przez system tokenów **JWT**, a hasła są szyfrowane przy użyciu **BCrypt**. Projekt stosuje podejście Clean Architecture z wykorzystaniem interfejsów i repozytoriów.
- **Baza danych:** Relacyjna baza danych. Skonfigurowana z wykorzystaniem systemu migracji w Entity Framework.
- **Aplikacja (Klient):** Desktopowa i mobilna aplikacja stworzona z wykorzystaniem frameworka **.NET MAUI**. Pozwala ona na instalację na systemach Windows oraz Android z jednego kodu źródłowego.

2.2. Wzorzec MVVM w aplikacji klienckiej

Aby oddzielić widok od logiki programu w aplikacji MAUI, zastosowano popularny wzorzec projektowy **MVVM (Model-View-ViewModel)**:

- **Widoki (Views):** To pliki typu XAML zawierające wyłącznie strukturę ekranu (np. guziki, pola tekstowe). Nie ma tu logiki biznesowej.
- **Modele Widoków (ViewModels):** Klasy w języku C#, które przygotowują i trzymają dane potrzebne do wyświetlenia na ekranie. Komunikują się z plikiem XAML za pomocą tzw. bindowania (Binding).
- **Serwisy:** Pomocnicze klasy (np. AuthService, ExpenseService), które odpowiadają za pobieranie danych ze zdalnego serwera API.

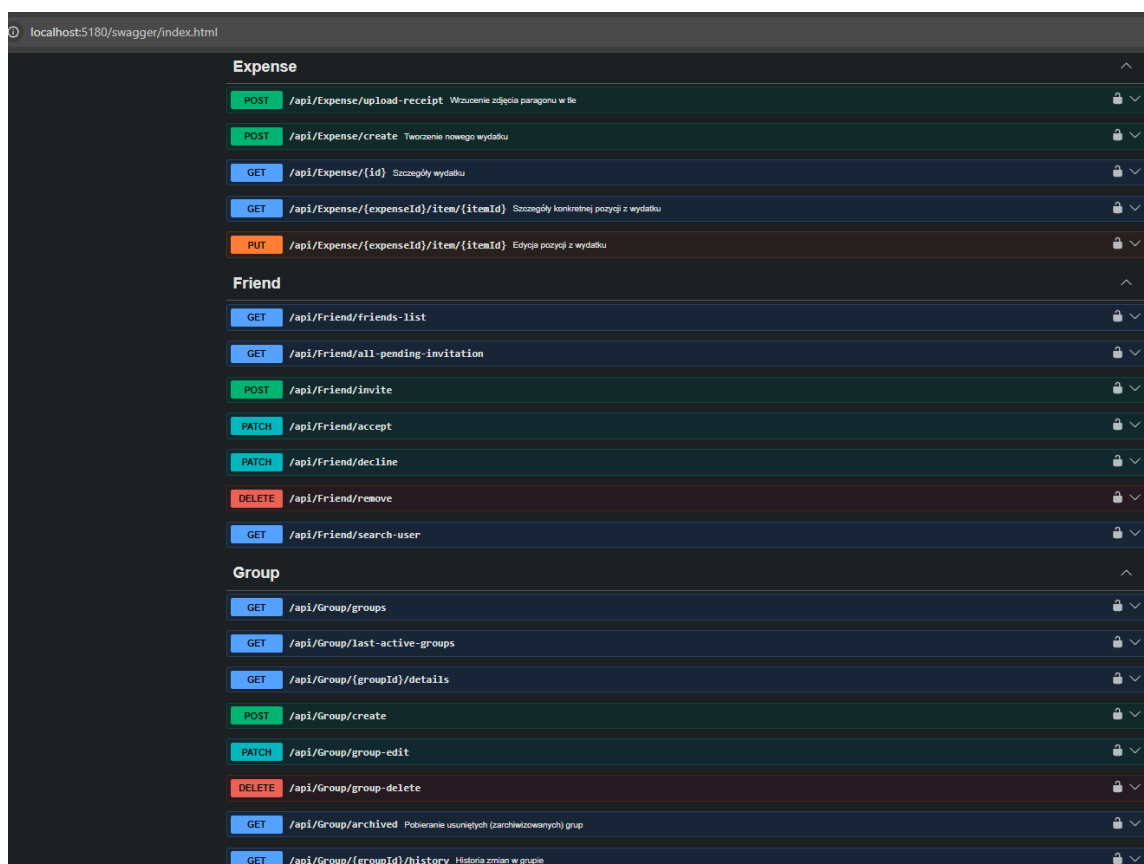


Rysunek 1: Drzewo projektu w środowisku Visual Studio.

3. Specyfikacja REST API

Aplikacja .NET MAUI działa jako bezstanowy klient. Cała logika biznesowa wykonywana jest bezpiecznie po stronie serwera API. Komunikacja odbywa się z wykorzystaniem protokołu HTTP, a wymiana informacji bazuje na obiektach w formacie JSON (Data Transfer Objects). Dostęp do większości zasobów wymaga nagłówka `Authorization: Bearer <token>`.

Dzięki zastosowaniu standardu OpenAPI i potężnego narzędzia **Swagger UI**, cała interaktywna dokumentacja punktów końcowych (wraz z wylistowanymi wszystkimi modelami DTO) jest dostępna graficznie z poziomu przeglądarki pod adresem `/swagger`. Pozwala to na wygodne testowanie żądań bez konieczności używania zewnętrznych narzędzi typu Postman.



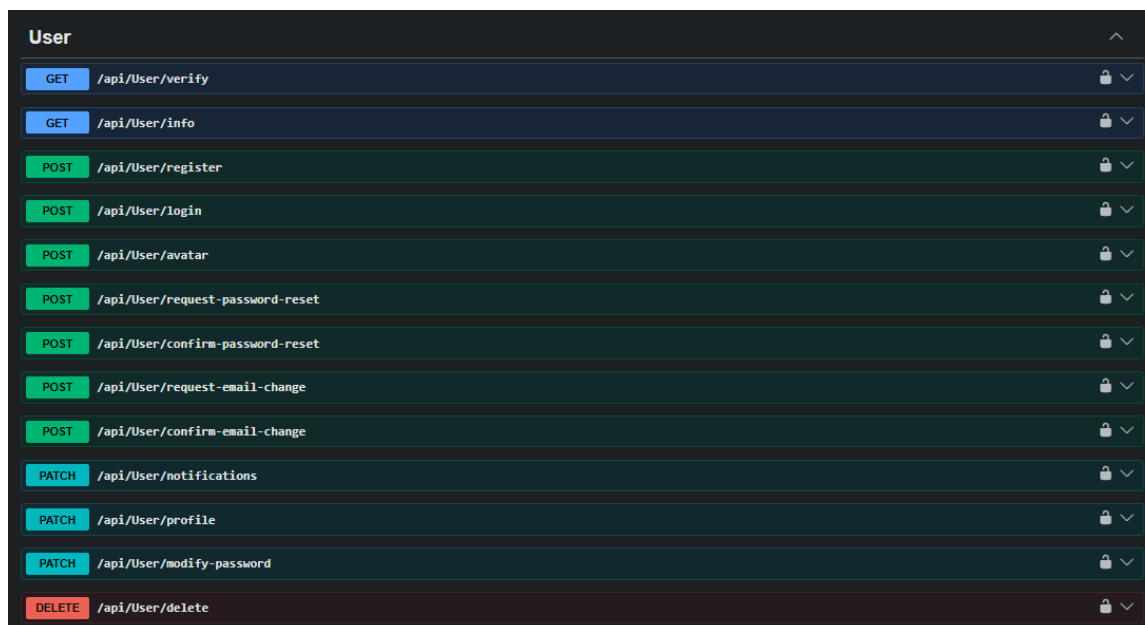
Rysunek 2: Interfejs Swagger API – część 1. Zestawienie podstawowych operacji CRUD dla domen: **Expense** (zarządzanie paragonami), **Friend** (sieć znajomych) oraz **Group** (środowiska współdzielone).

GroupMember			^
GET	/api/GroupMember/all-pending-invitation		🔒
GET	/api/GroupMember/{groupId}/all-group-members		🔒
POST	/api/GroupMember/invite		🔒
PATCH	/api/GroupMember/accept		🔒
PATCH	/api/GroupMember/decline		🔒
PATCH	/api/GroupMember/update-role		🔒
PATCH	/api/GroupMember/{groupId}/set-fav		🔒
DELETE	/api/GroupMember/{groupId}/leave		🔒
DELETE	/api/GroupMember/{groupId}/kick/{targetUserId}		🔒
Notification			^
GET	/api/Notification/get-all-notifications		🔒
GET	/api/Notification/get-last-5-notifications		🔒
PATCH	/api/Notification/set-as-read		🔒
PATCH	/api/Notification/set-all-as-read		🔒
DELETE	/api/Notification/delete		🔒
DELETE	/api/Notification/delete-all		🔒
Seeder			^
POST	/api/Seeder/heavy-login-seed		🔒

Rysunek 3: Interfejs Swagger API – część 2. Punkty końcowe operujące na relacjach: **Group-Member** (role, uprawnienia), **Notification** (system powiadomień) oraz **Seeder** (wstrzykiwanie danych testowych).

Settlement			^
GET	/api/Settlement/get-user-incomes-summ		🔒
GET	/api/Settlement/get-user-expenses-summ		🔒
GET	/api/Settlement/get-user-expense-history		🔒
GET	/api/Settlement/current-debt		🔒
GET	/api/Settlement/payable-options		🔒
POST	/api/Settlement/create		🔒
POST	/api/Settlement/create-net-pay		🔒
POST	/api/Settlement/accept/{id}		🔒
POST	/api/Settlement/reject/{id}		🔒
POST	/api/Settlement/accept-net/{senderId}		🔒
POST	/api/Settlement/reject-net/{senderId}		🔒
POST	/api/Settlement/remind-debt		🔒
POST	/api/Settlement/compensate		🔒

Rysunek 4: Interfejs Swagger API – część 3. Główny silnik finansowy reprezentowany przez sekcję **Settlement**. Odpowiada za cały obieg pieniądza, deklaracje przelewów i kompensatę grafu zobowiązań.



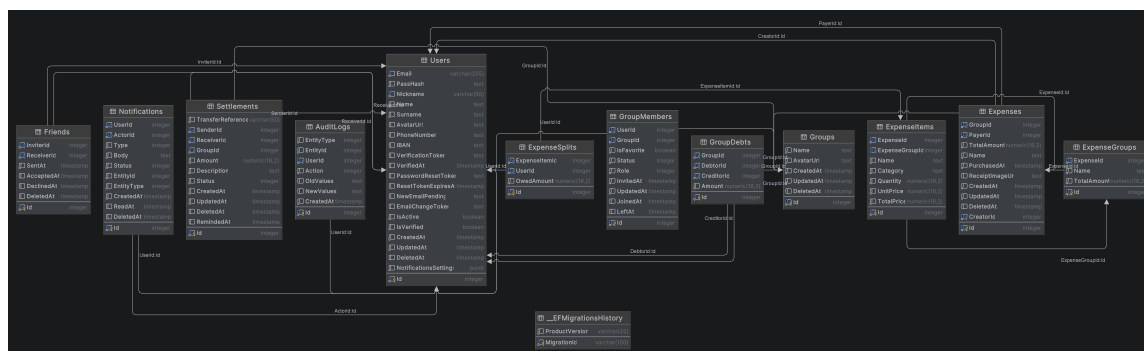
User		
GET	/api/User/verify	🔒
GET	/api/User/info	🔒
POST	/api/User/register	🔒
POST	/api/User/login	🔒
POST	/api/User/avatar	🔒
POST	/api/User/request-password-reset	🔒
POST	/api/User/confirm-password-reset	🔒
POST	/api/User/request-email-change	🔒
POST	/api/User/confirm-email-change	🔒
PATCH	/api/User/notifications	🔒
PATCH	/api/User/profile	🔒
PATCH	/api/User/modify-password	🔒
DELETE	/api/User/delete	🔒

Rysunek 5: Interfejs Swagger API – część 4. Moduł autoryzacji **User**. Punkty końcowe obsługujące rejestrację, uwierzytelnianie (JWT) oraz modyfikację wrażliwych danych konta.

4. Opis Bazy Danych

4.1. Schemat bazy (Diagram ERD)

Baza danych składa się w sumie z 11 tabel połączonych odpowiednimi relacjami. Całość została zaprojektowana zgodnie z podejściem Code-First (baza generowana bezpośrednio z klas C#).



Rysunek 6: Kompletny diagram związków encji (ERD) obrazujący architekturę relacyjnej bazy danych systemu PayItOff. Widoczne powiązania kluczy obcych i tabele łącznikowe.

4.2. Opis poszczególnych tabel

Users	
Email	varchar(255)
PassHash	text
Nickname	varchar(50)
Name	text
Surname	text
AvatarUrl	text
PhoneNumber	text
IBAN	text
VerificationToken	text
VerifiedAt	timestamp
PasswordResetToken	text
ResetTokenExpiresAt	timestamp
NewEmailPending	text
EmailChangeToken	text
IsActive	boolean
IsVerified	boolean
CreatedAt	timestamp
UpdatedAt	timestamp
DeletedAt	timestamp
NotificationsSettings	jsonb
Id	integer

Friends	
InviterId	integer
ReceiverId	integer
SentAt	timestamp
AcceptedAt	timestamp
DeclinedAt	timestamp
DeletedAt	timestamp
Id	integer

Rysunek 7: Struktura tabel **Users** oraz **Friends**. Po lewej główny agregat tożsamości z ustawieniami powiadomień. Po prawej tabela łącznikowa realizująca relację sieci kontaktów (Many-to-Many).

Groups	
Name	text
AvatarUrl	text
CreatedAt	timestamp
UpdatedAt	timestamp
DeletedAt	timestamp
Id	integer

GroupMembers	
UserId	integer
GroupId	integer
IsFavorite	boolean
Status	integer
Role	integer
InvitedAt	timestamp
UpdatedAt	timestamp
JoinedAt	timestamp
LeftAt	timestamp
Id	integer

Rysunek 8: Struktura środowisk współdzielonych. Tabela **Groups** realizuje miękkie usuwanie (DeletedAt), a **GroupMembers** odpowiada za kontrolę dostępu i przypisywanie ról (RBAC).

Expenses	
GroupId	integer
PayerId	integer
TotalAmount	numeric(18,2)
Name	text
PurchasedAt	timestamp
ReceiptImageUrl	text
CreatedAt	timestamp
UpdatedAt	timestamp
DeletedAt	timestamp
CreatorId	integer
Id	integer

ExpenseItems	
ExpenseId	integer
ExpenseGroupId	integer
Name	text
Category	text
Quantity	numeric(18,2)
UnitPrice	numeric(18,2)
TotalPrice	numeric(18,2)
Id	integer

Rysunek 9: Modelowanie rachunków. Tabela **Expenses** przechowuje globalne metadane wydatku, natomiast **ExpenseItems** pozwala na szczegółowe rozbicie paragonu na pojedyncze produkty.

ExpenseSplits	
ExpenseItemId	integer
UserId	integer
OwedAmount	numeric(18,2)
Id	integer

ExpenseGroups	
ExpenseId	integer
Name	text
TotalAmount	numeric(18,2)
Id	integer

Rysunek 10: Tabela **ExpenseSplits** to kluczowy punkt obliczeniowy dla przydziału należności ułamkowych (rozwiązanie Penny-Drop). **ExpenseGroups** wspomaga analitykę i kategoryzację.

GroupDebts	
GroupId	integer
DebtorId	integer
CreditorId	integer
Amount	numeric(18,2)
Id	integer

Settlements	
TransferReference	varchar(50)
SenderId	integer
ReceiverId	integer
GroupId	integer
Amount	numeric(18,2)
Description	text
Status	integer
CreatedAt	timestamp
UpdatedAt	timestamp
DeletedAt	timestamp
RemindedAt	timestamp
Id	integer

Rysunek 11: Mechanizmy windykacyjne. **GroupDebts** to zdenormalizowany bufor odciążający bazę danych przy wyliczaniu grafu. **Settlements** zabezpiecza transakcje i transfery wyrównujące.

Notifications	
 UserId	integer
 ActorId	integer
 Type	integer
 Body	text
 Status	integer
 EntityId	integer
 EntityType	integer
 CreatedAt	timestamp
 ReadAt	timestamp
 DeletedAt	timestamp
 Id	integer

AuditLogs	
 EntityType	integer
 EntityId	integer
 UserId	integer
 Action	integer
 OldValues	text
 NewValues	text
 CreatedAt	timestamp
 Id	integer

Rysunek 12: Zdarzenia i bezpieczeństwo. **Notifications** utrzymuje stan odczytu asynchronicznych powiadomień. **AuditLogs** realizuje wzorzec Audit Trail logując wszystkie historyczne modyfikacje.

5. Wygląd i przepływ w aplikacji (.NET MAUI)

Poniższa sekcja prezentuje interfejs graficzny aplikacji zreorganizowany w pełni chronologicznie. Pokazuje ona naturalną ścieżkę użytkownika (User Journey) – od pierwszego uruchomienia aplikacji i budowania sieci kontaktów, przez tworzenie grup i wprowadzanie kosztów, aż po ostateczne rozliczenia długów oraz zarządzanie retencją danych (usuwanie profilu).

5.1. Krok 1: Inicjacja (Rejestracja i Logowanie)

Proces rozpoczyna się od utworzenia nowego profilu w systemie, weryfikacji haseł, po czym następuje autoryzacja z użyciem zwalidowanych poświadczeń.

Stwórz konto

Email
email@przyklad.pl

Nazwa użytkownika (Nick)
TwójNick

Imię
Jan

Nazwisko
Kowalski

Hasło
Min. 8 znaków, wielka litera, cyfra i znak specjalny
.....

Numer telefonu
+48 123 456 789

Numer konta (IBAN)
PL 00 0000 0000...

Kliknij, aby wybrać zdjęcie

Zarejestruj się

Masz już konto? Zaloguj się

Stwórz konto

Email
JanKowalski@poczta.pl

Nazwa użytkownika (Nick)
JanKowalski

Imię
Jan

Nazwisko
Kowalski

Hasło
Min. 8 znaków, wielka litera, cyfra i znak specjalny
.....

Numer telefonu
+48 123 123 123

Numer konta (IBAN)
PL 00 0000 0000...

Kliknij, aby wybrać zdjęcie

Zarejestruj się

Masz już konto? Zaloguj się

Rysunek 13: Kolejne etapy formularza rejestracji konta użytkownika. Aplikacja zbiera i podaje ścisłej walidacji podstawowe dane uwierzytelniające.



Stwórz konto

Email
JanKowalski@poczta.pl

Nazwa użytkownika (Nick)
JanKowalski

Imię
Jan

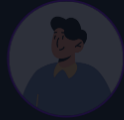
Nazwisko
Kowalski

Sukces

Konto założone! Potwierdź rejestrację na mailu aby móc się zalogować...

OK

Numer konta (IBAN)
PL 00 0000 0000 ...



Tędy, aby wybrać zdjęcie

Zarejestruj się

This is a registration form titled "Stwórz konto". It has a dark background. The form contains several input fields: "Email" (filled with JanKowalski@poczta.pl), "Nazwa użytkownika (Nick)" (filled with JanKowalski), "Imię" (filled with Jan), "Nazwisko" (filled with Kowalski), and "Numer konta (IBAN)" (filled with PL 00 0000 0000 ...). Below the "Nazwisko" field is a success message box with a green border and a green background, containing the text "Sukces" and "Konto założone! Potwierdź rejestrację na mailu aby móc się zalogować...". Below this is a green button labeled "OK". At the bottom of the form is a button labeled "Zarejestruj się".

Rysunek 14: Proces weryfikacji e-mail. Widoczna symulacja skrzynki pocztowej (Mailtrap) z tokenem aktywacyjnym oraz ekran kończący rejestrację.

PayItOff!

Adres E-Mail lub Nazwa Użytkownika
JanKowalski

Hasło
.....

Zaloguj się

Nie masz konta? Zarejestruj się

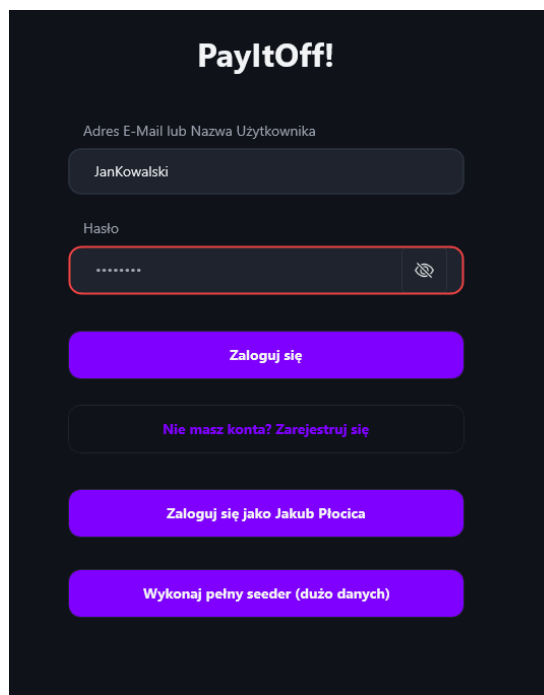
Zaloguj się jako Jakub Płocica

Wykonaj pełny seeder (dużo danych)

This is a login form titled "PayItOff!". It has a dark background. The form contains several input fields: "Adres E-Mail lub Nazwa Użytkownika" (filled with JanKowalski) and "Hasło" (filled with dots). Below the "Hasło" field is a button labeled "Zaloguj się". Below this is a button labeled "Nie masz konta? Zarejestruj się". Below this is a button labeled "Zaloguj się jako Jakub Płocica". Below this is a button labeled "Wykonaj pełny seeder (dużo danych)".

Hasło
JanKowalski123!

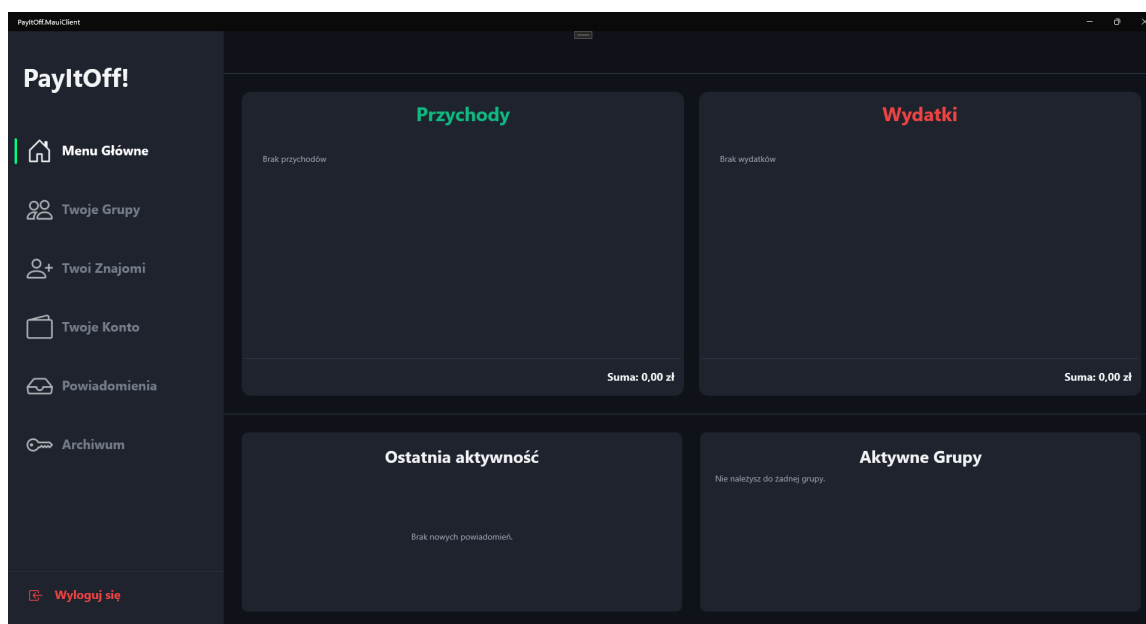
This is a password input field. It has a dark background. The input field is filled with the text "JanKowalski123!". To the right of the input field is an eye icon.

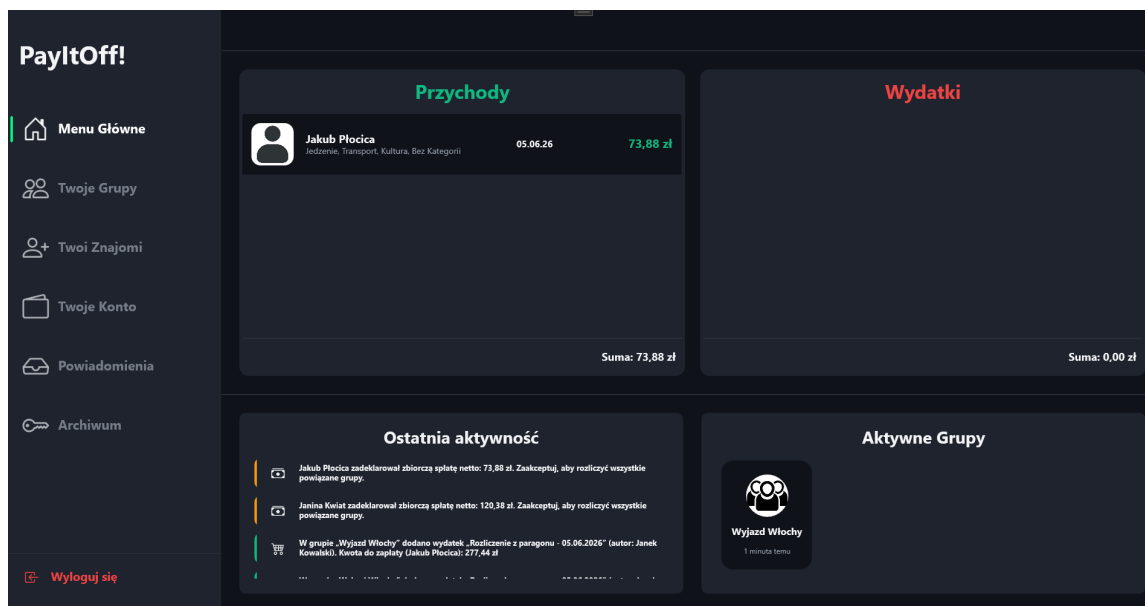
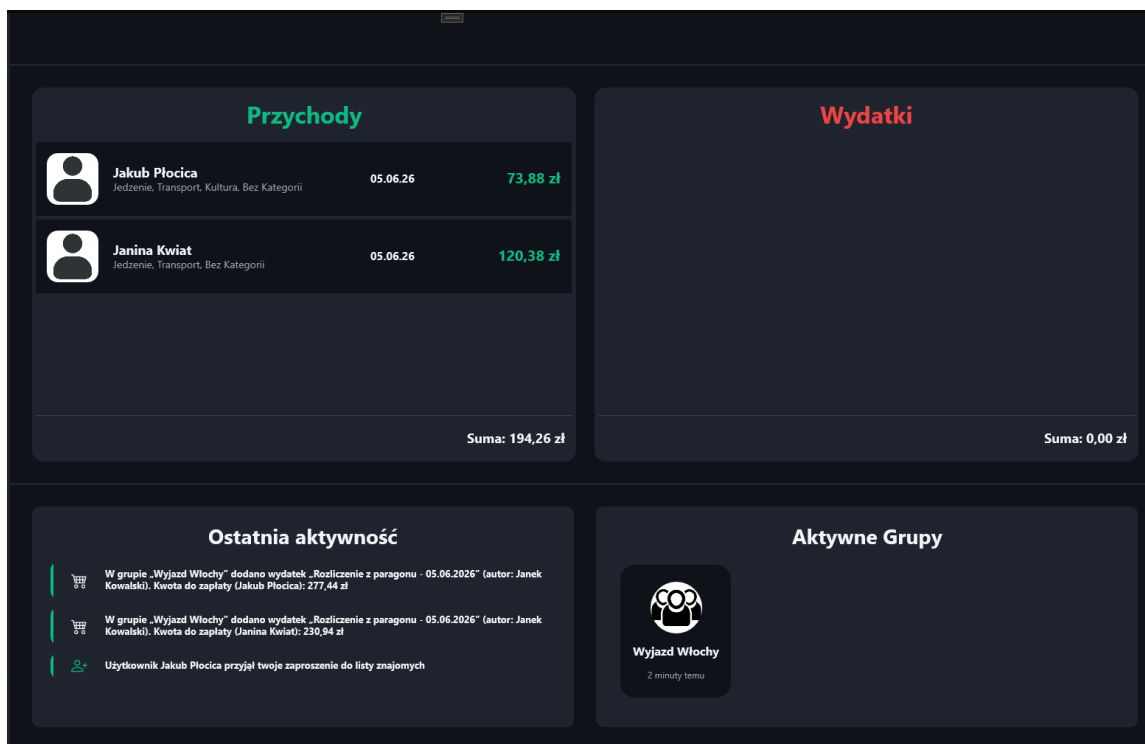


Rysunek 15: Moduł logowania. Widok z ukrytym hasłem, funkcja podglądu hasła w locie oraz mechanizm walidacji nieprawidłowych poświadczeń autoryzacyjnych.

5.2. Krok 2: Konfiguracja, Nawigacja i Zarządzanie Kontem

Po pomyślnej autoryzacji system przekierowuje nas do głównego pulpitu. Otwiera się również dostęp do menu nawigacyjnego i edycji profilu (zmiana awatara, haseł, e-maila oraz usunięcie konta).





Rysunek 16: Główny pulpit nawigacyjny (Dashboard). Od lewej: pusty stan początkowy, stan z aktywnością wydatkową oraz widok reagujący na spłaty i odrzucenia (zmiany na liście powiadomień).

PayItOff!



Menu Główne



Twoje Grupy



Twoi Znajomi



Twoje Konto



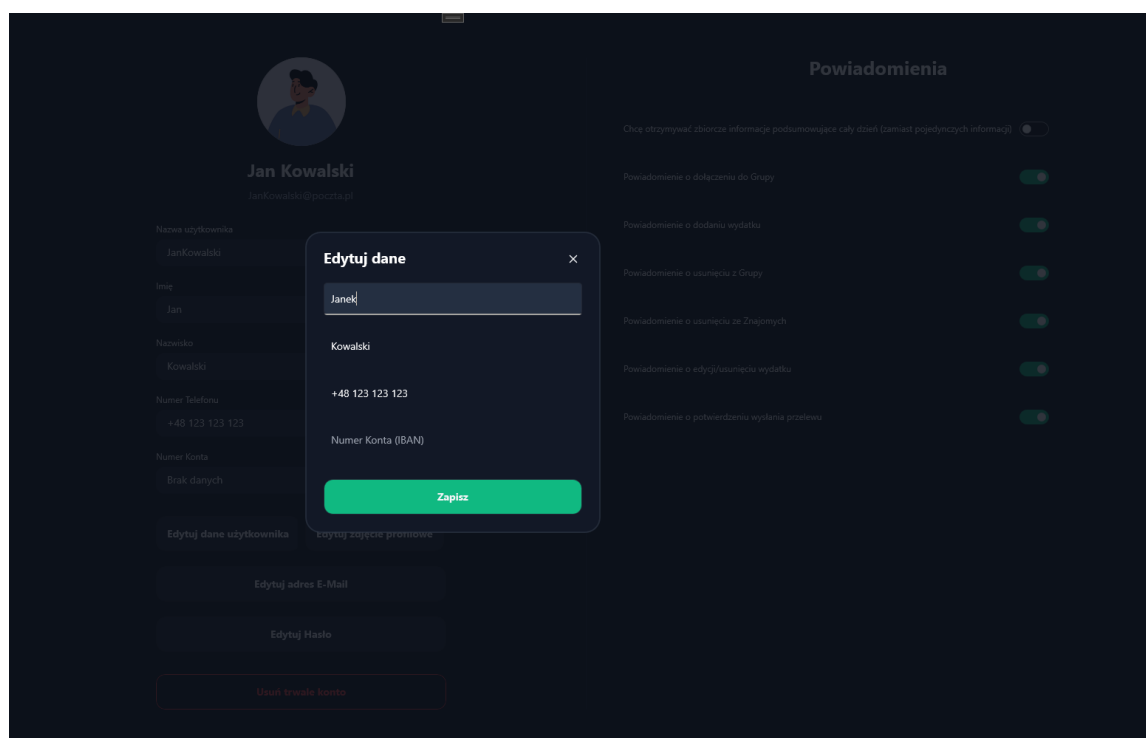
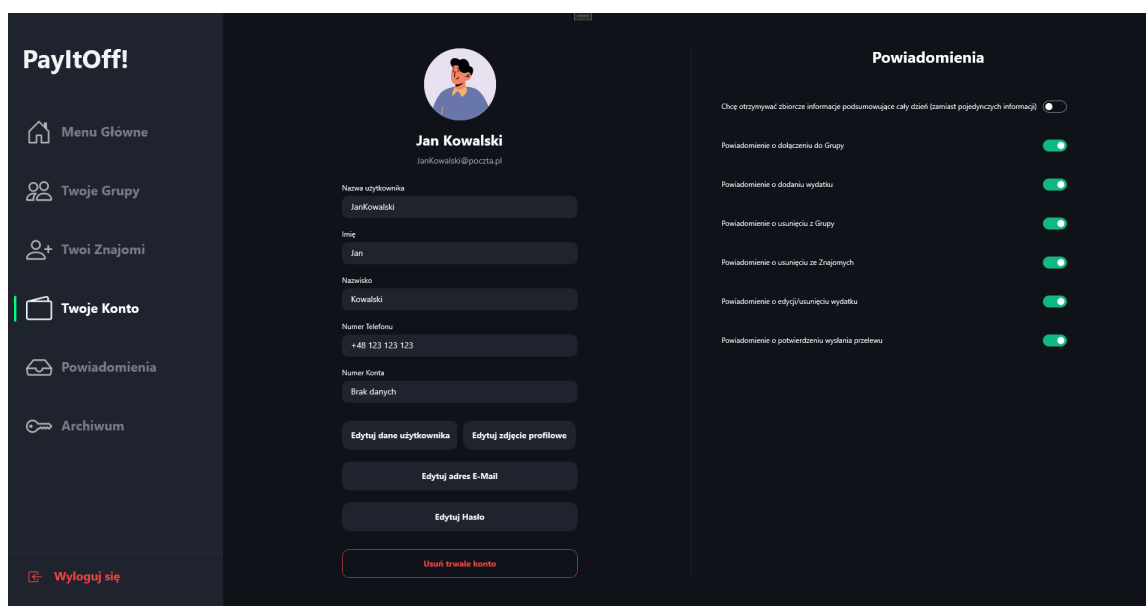
Powiadomienia



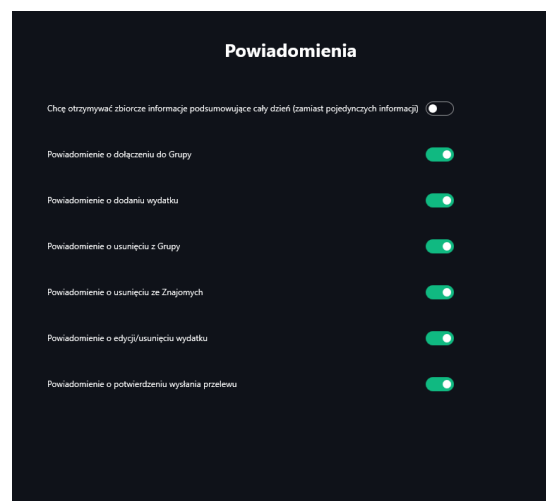
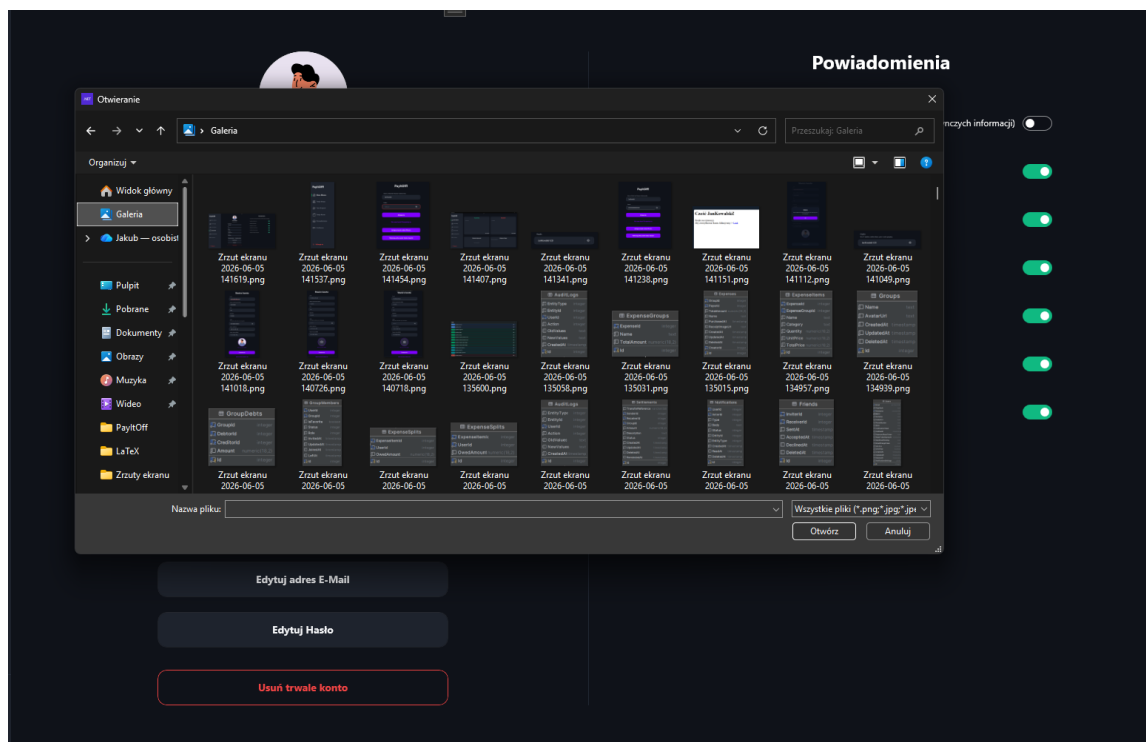
Archiwum



Wyloguj się

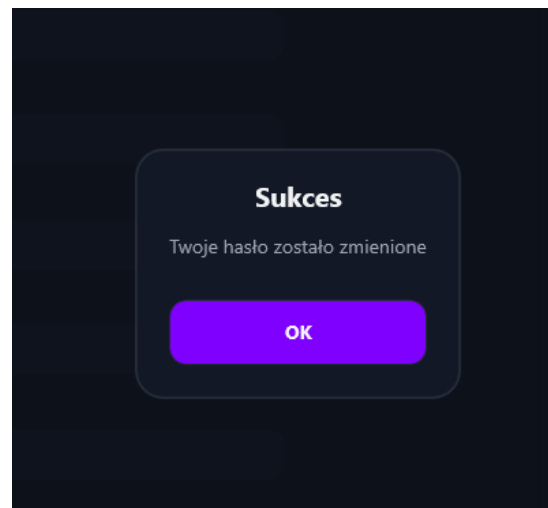


Rysunek 17: Rozwijane menu boczne (Sidebar) umożliwiające szybką nawigację oraz główny widok profilu z funkcją edycji podstawowych danych osobowych.



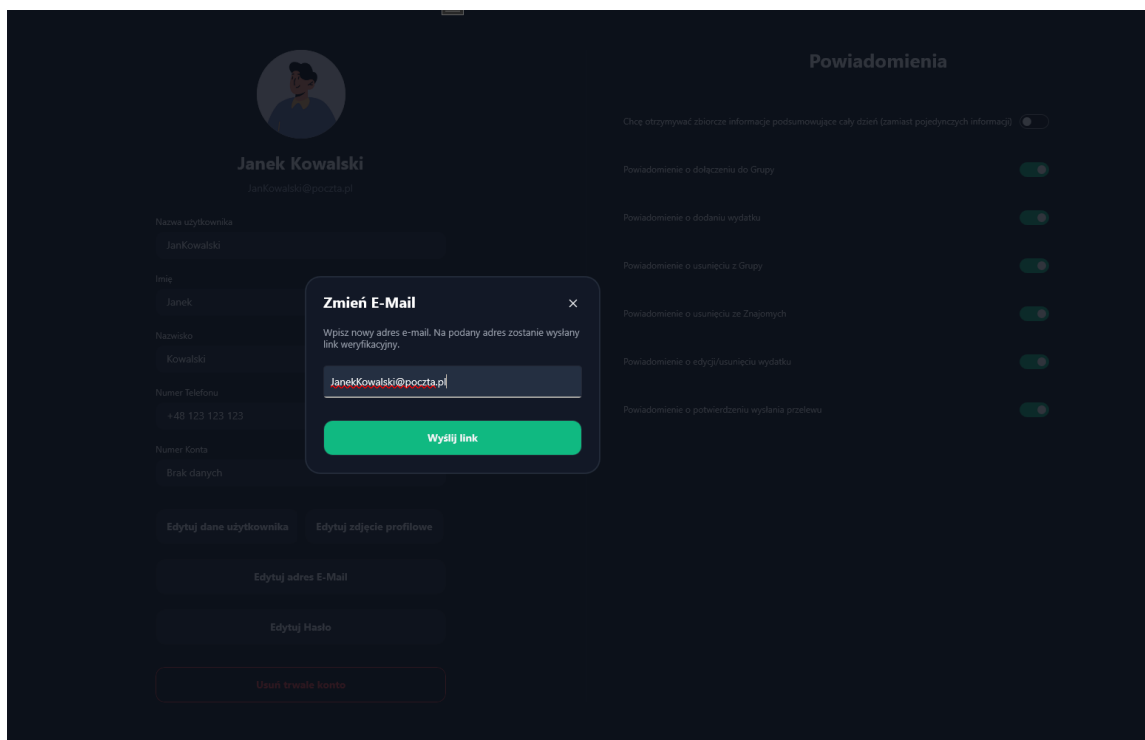
Rysunek 18: Modyfikacja profilu. Przebieg zmiany zdjęcia profilowego (awatara), potwierdzenie wgrania danych oraz panel konfiguracji globalnych preferencji dla systemu notyfikacji.

The screenshot shows a user profile for 'Janek Kowalski' with email 'JanekKowalski@poczta.pl'. The profile details include: Nazwa użytkownika: JanKowalski, Imię: Janek, Nazwisko: Kowalski, Numer Telefonu: +48 123 123 123, Numer Konta: Brak danych. A modal window titled 'Zmień Hasło' is open, featuring two password input fields with masked characters and a green 'Zapisz' (Save) button.

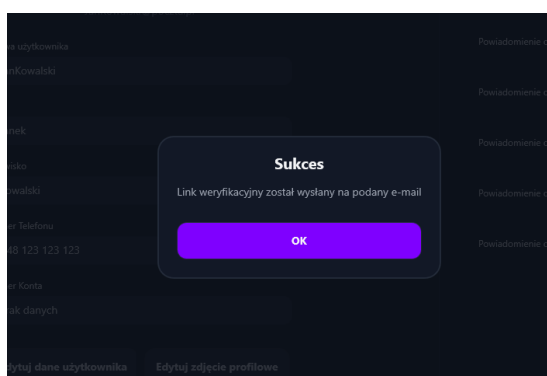


The screenshot shows the same user profile for 'Janek Kowalski'. Below the profile details, there are links: 'Edytuj dane użytkownika', 'Edytuj zdjęcie profilowe', 'Edytuj adres E-Mail', and 'Edytuj Hasło'. A modal window titled 'Usuń Konto' is open, displaying a warning: 'Czy na pewno chcesz usunąć to konto? Tej operacji nie można cofnąć, stracisz wszystkie dane!'. The modal has two buttons: 'Anuluj' (Cancel) and 'Usuń Trwale' (Delete Permanently).

Rysunek 19: Krytyczne operacje na profilu. Bezpieczna procedura zmiany hasła uwierzytelniającego oraz wywołanie formularza usunięcia konta (wiążące się z weryfikacją zależności rozliczeniowych).



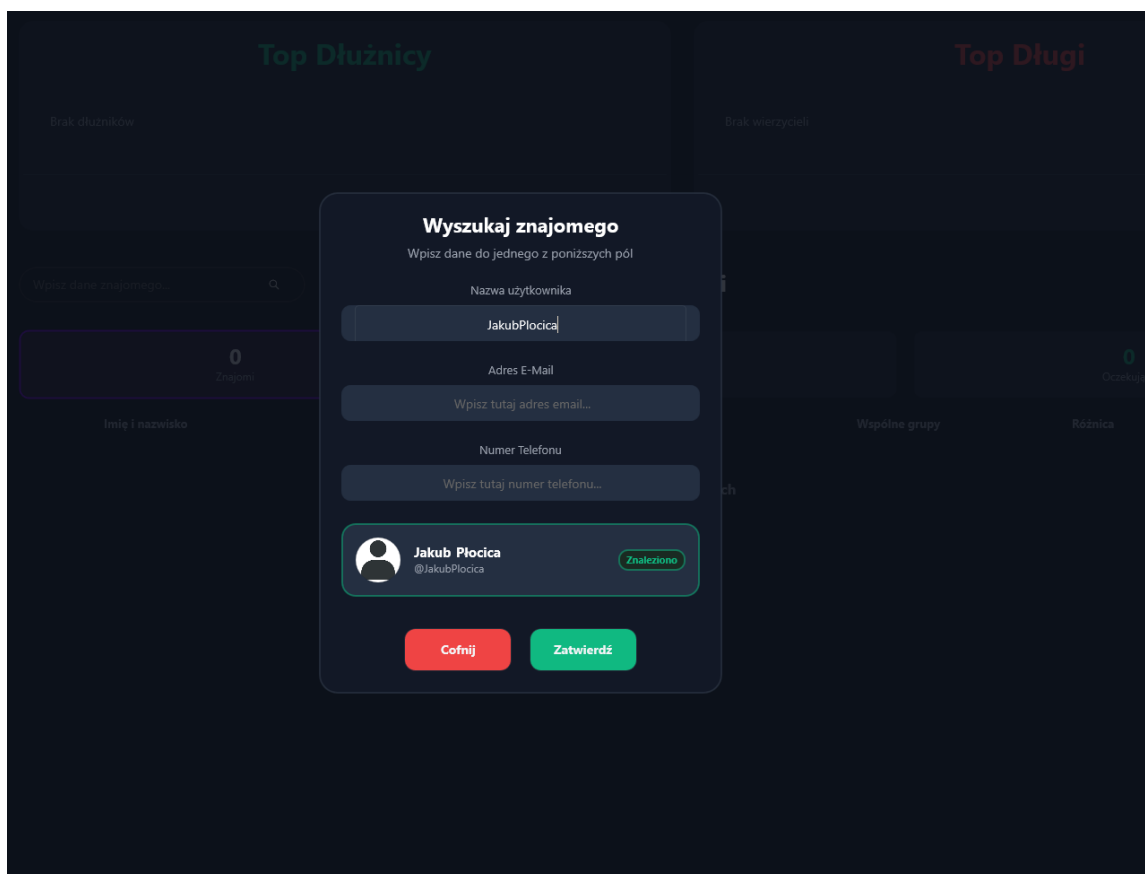
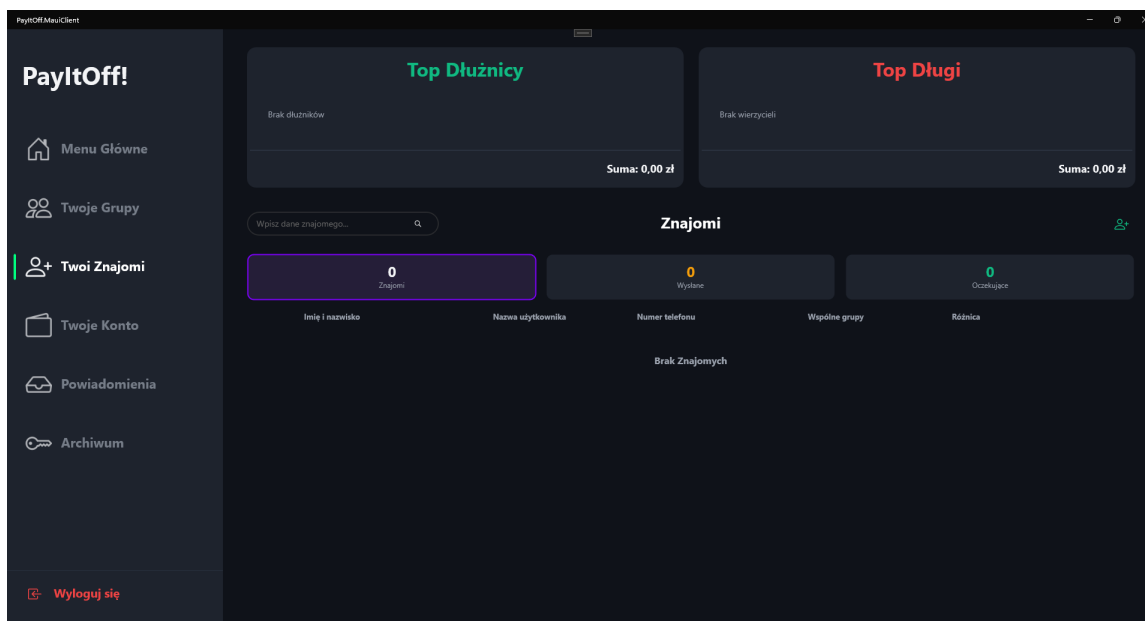
Rysunek 20: Procedura modyfikacji adresu e-mail powiązana z wymogiem potwierdzenia dostępu. Skrzynka weryfikacyjna przechwytuje token zabezpieczający operację.

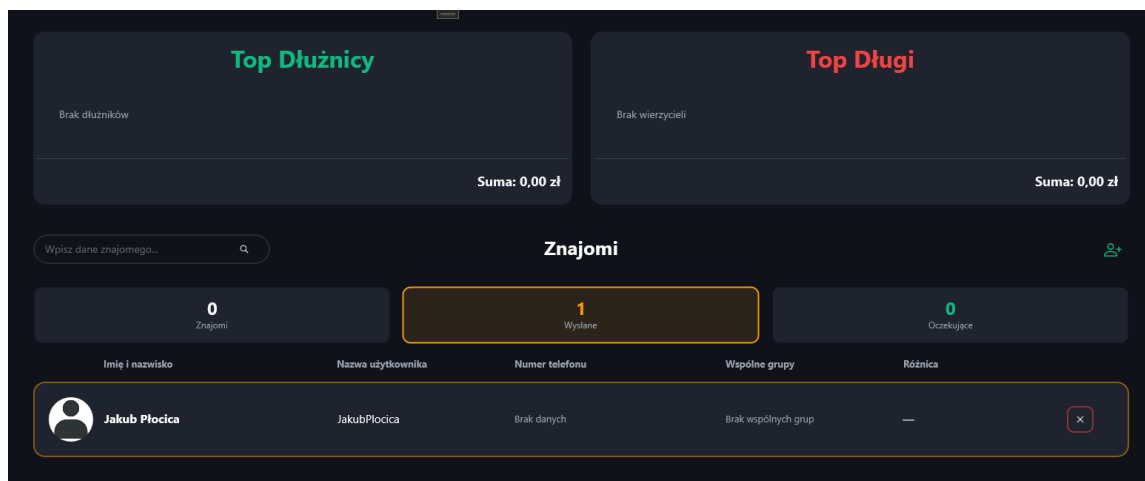


Rysunek 21: Zakończenie i zwalidowanie zmiany adresu e-mail, co natychmiast odświeża kontekst tożsamości we wbudowanych tokenach JWT.

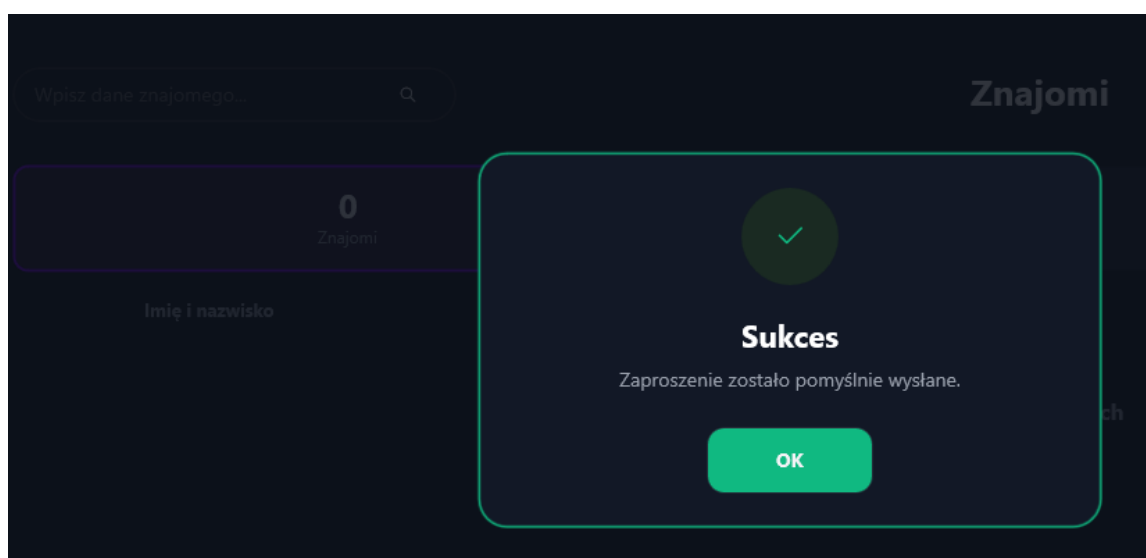
5.3. Krok 3: Budowanie Sieci Kontaktów (Friends)

Aplikacja zyskuje wartość dopiero w interakcji z innymi. Przed założeniem grupy, nawiązujemy relacje wyszukując znajomych i wysyłając im wirtualne zaproszenia do sieci.





Rysunek 22: Inicjacja sieci kontaktów. Pusty stan modułu znajomych, silnik wyszukiwania użytkowników po predykatkach (np. nazwa) oraz wgląd w zaproszenia oczekujące (Pending).



Top Dłużnicy

	Janina Kwiat	9 723,34 zł
	Zofia Wiśniewska	1 399,08 zł
		Suma: 11 122,42 zł

Top Długi

	Janina Kwiat	9 338,06 zł
	Zofia Wiśniewska	2 130,89 zł
		Suma: 11 468,95 zł

Znajomi

2
Znajomi

1
Wysłane

1
Oczekujące

Imię i nazwisko	Nazwa użytkownika	Numer telefonu	Wspólne grupy	Różnica	
Janek Kowalski	JanKowalski	Brak danych	Brak wspólnych grup	—	☒ ☐

Wszystkie 1

Nieprzeczytane 1

Wymaga akcji 0

Oznacz wszystkie jako przeczytane

Usuń wszystkie powiadomienia

Użytkownik Jakub Płocica przyjął twoje zaproszenie do listy znajomych

Teraz

Rysunek 23: Obsługa cyklu życia zaproszenia. Potwierdzenie wysłania, widok oczekującego zaproszenia u odbiorcy, i wygenerowanie notyfikacji w przypadku jego zaakceptowania.

Wszystkie 67

Nieprzeczytane 67

Wymaga akcji 1

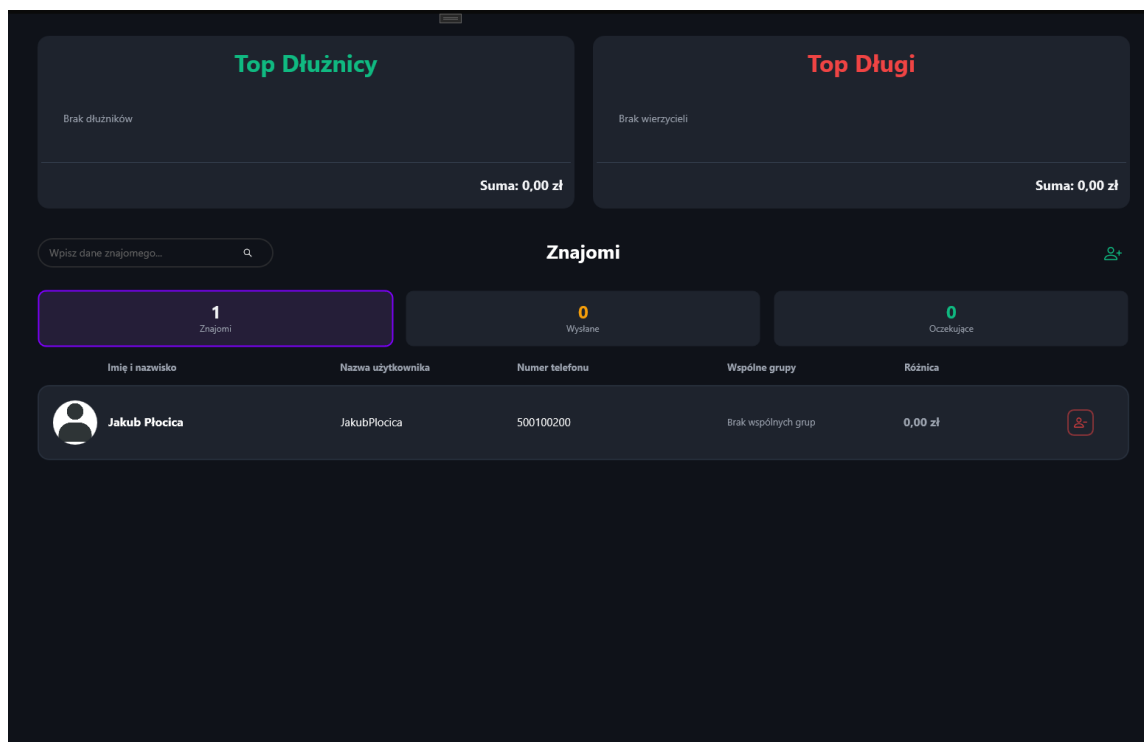
Oznacz wszystkie jako przeczytane

Usuń wszystkie powiadomienia

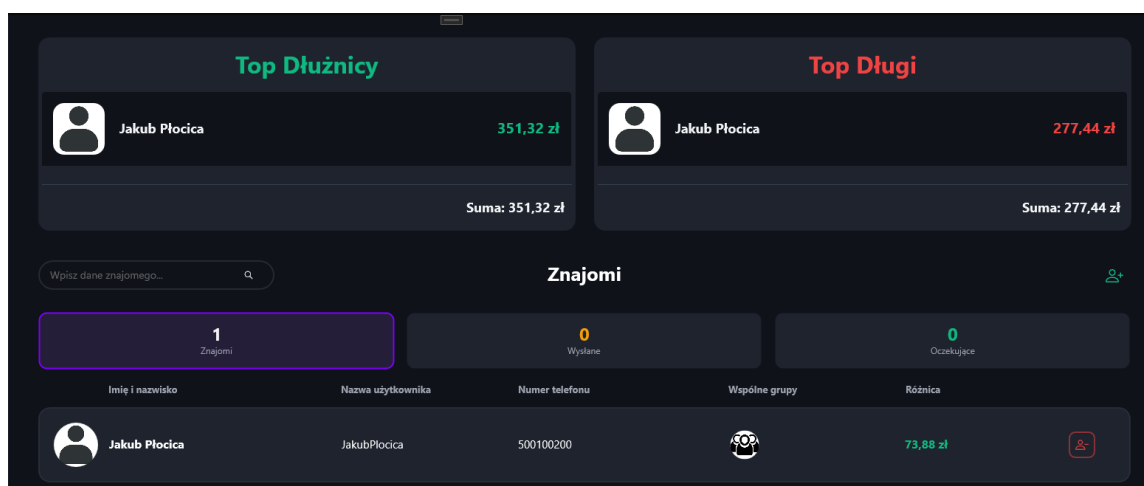
Janek Kowalski wysłał/wysłała Ci zaproszenie do znajomych.

6 minut temu

☒
☐
☐



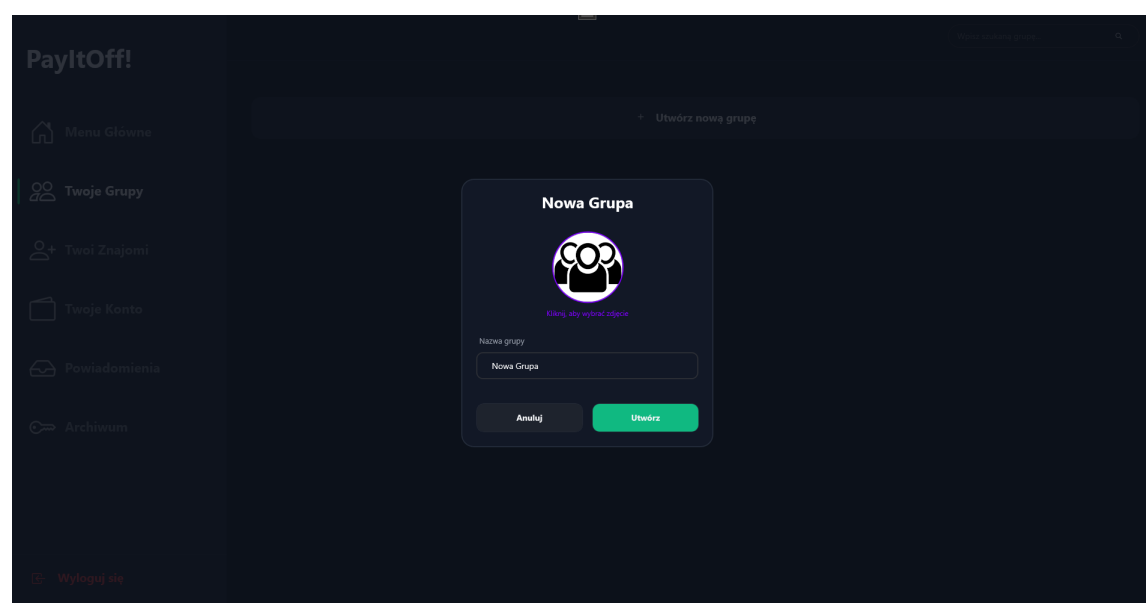
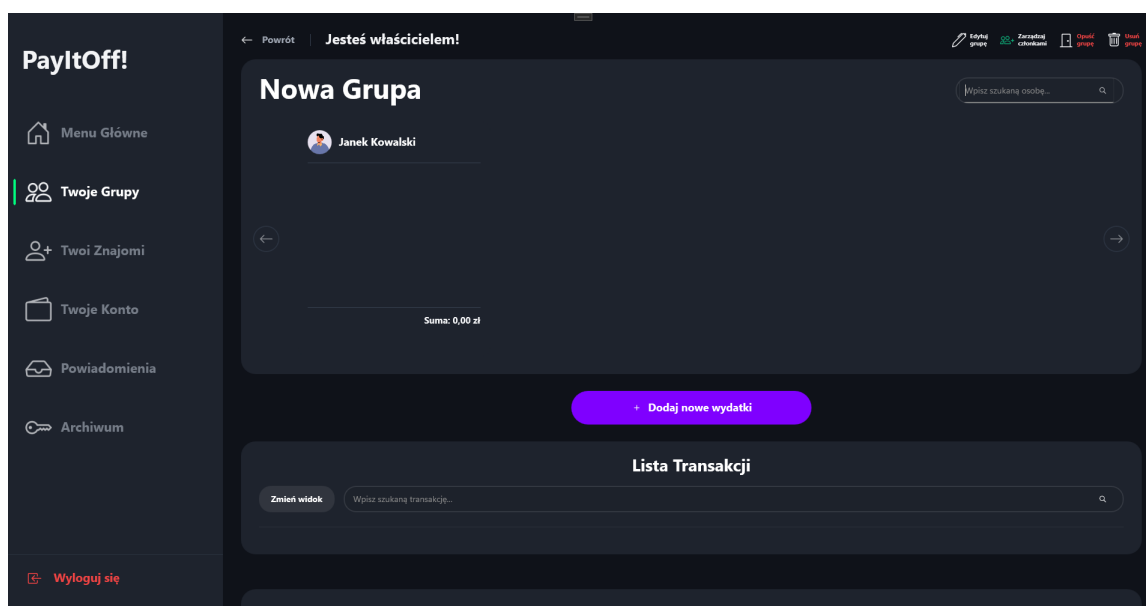
Rysunek 24: Finalizacja nawiązania relacji. Odbiorca otrzymuje potwierdzenie z serwera, a widok znajomych zostaje odświeżony bez konieczności przeładowywania aplikacji.



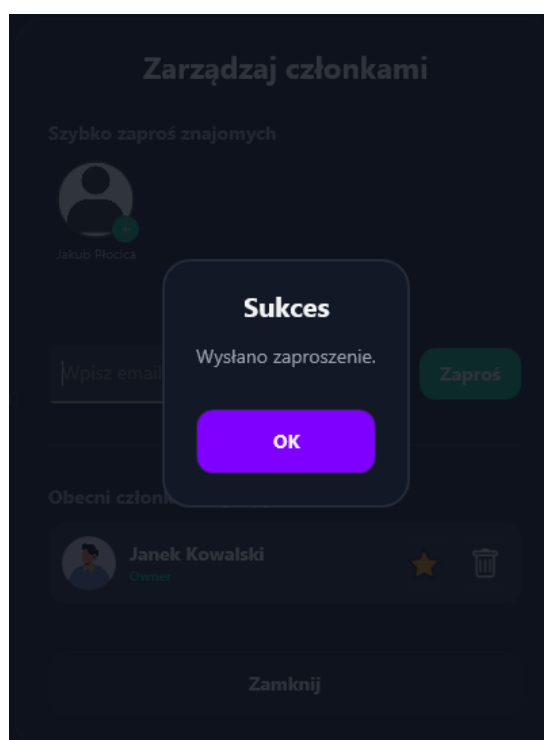
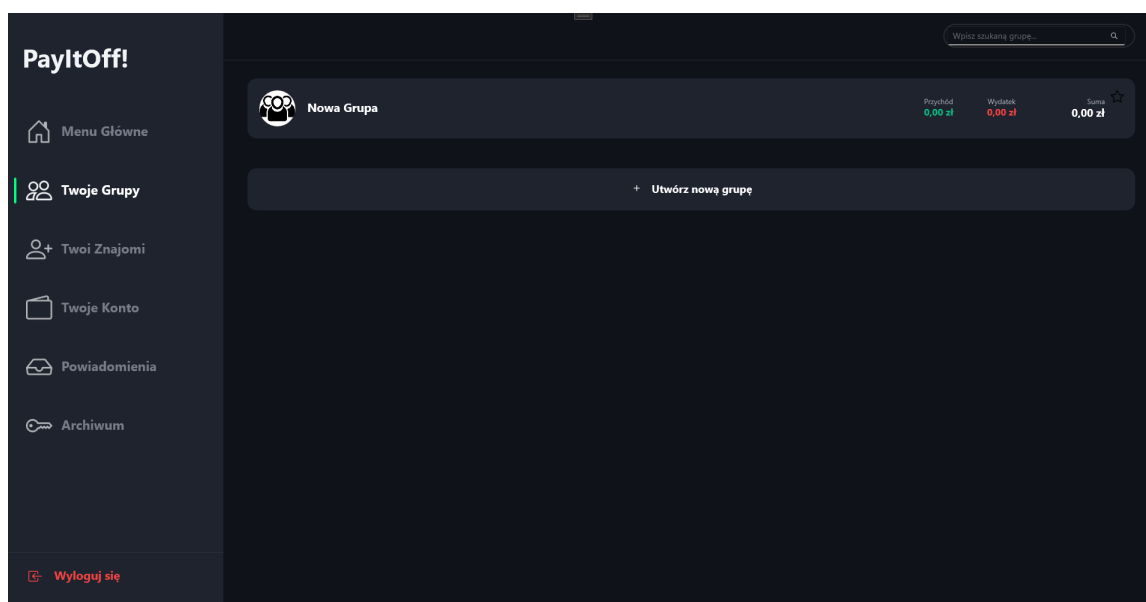
Rysunek 25: Moduł grywalizacji / statystyk. Widoczny ranking powiązany ze znajomymi uwzględniający historię poniesionych wydatków po uprzednim ich zaksięgowaniu.

5.4. Krok 4: Tworzenie Ekosystemu (Grupy Wydatków)

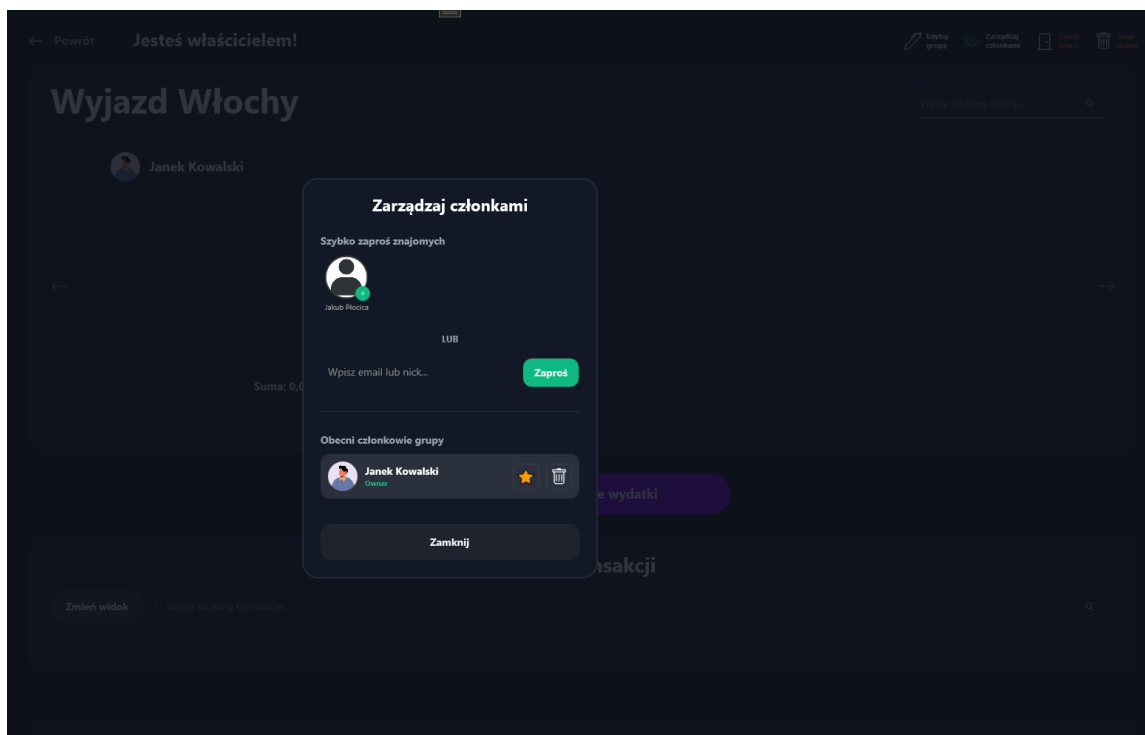
Po rozbudowaniu sieci kontaktów, tworzymy wspólne pokoje (Grupy). To w nich gromadzą się uczestnicy, zyskując uprawnienia (Admin/Member) i przypisując sobie role do późniejszego rozliczania.



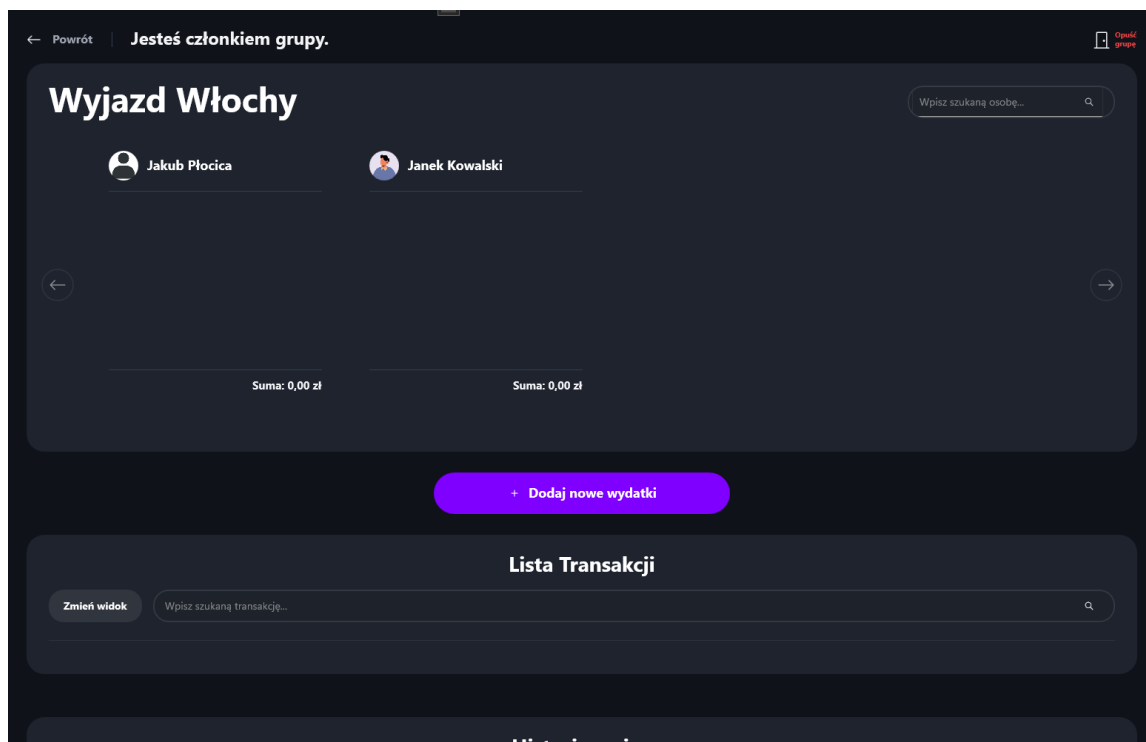
Rysunek 26: Inicjacja grupy rozliczeniowej. Po lewej ekran startowy, po prawej formularz tworzenia nowego środowiska współdzielonego z przypisywaniem nazwy.



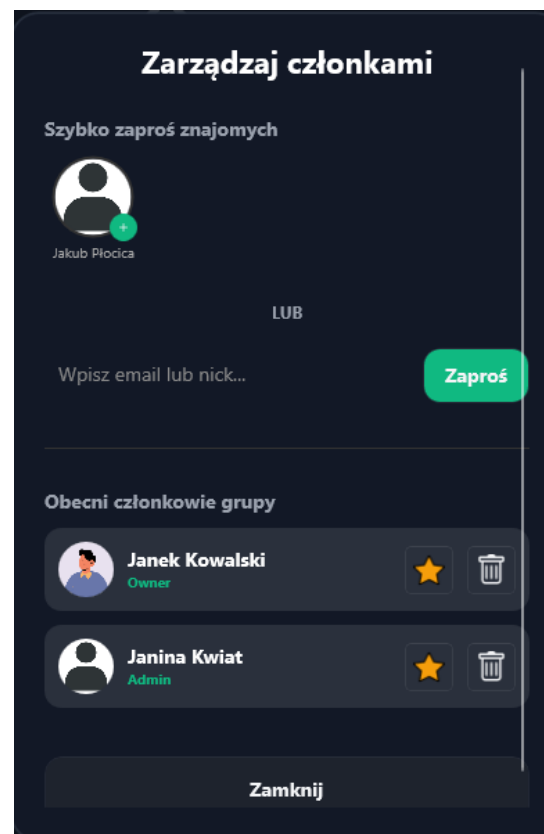
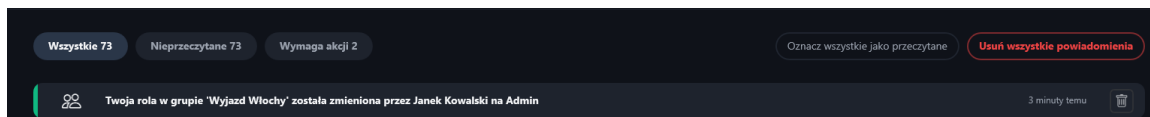
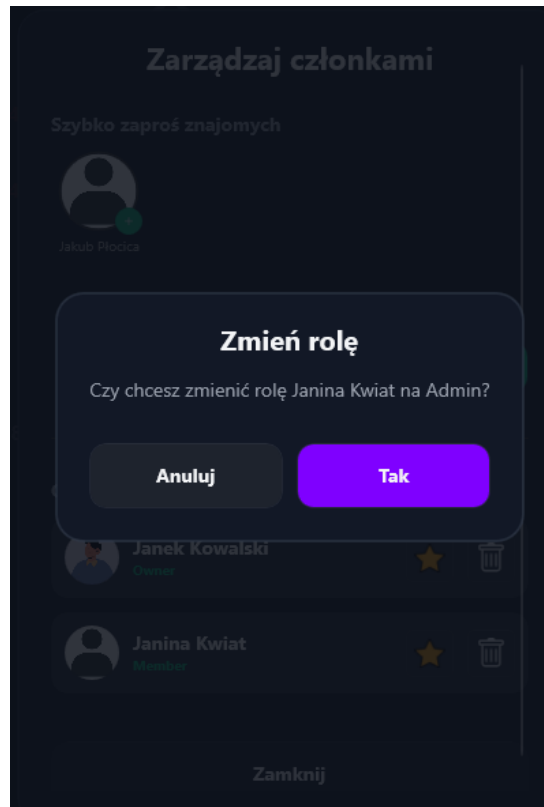
Rysunek 27: Widok nowo powstałej grupy oraz wykorzystanie dedykowanego modułu szybkiego zapraszania znajomych do współpracy.



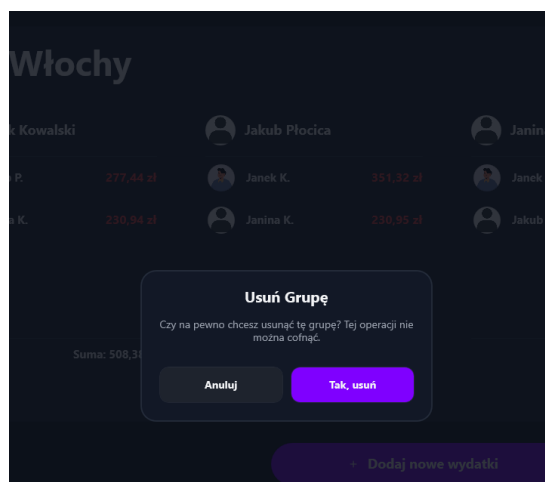
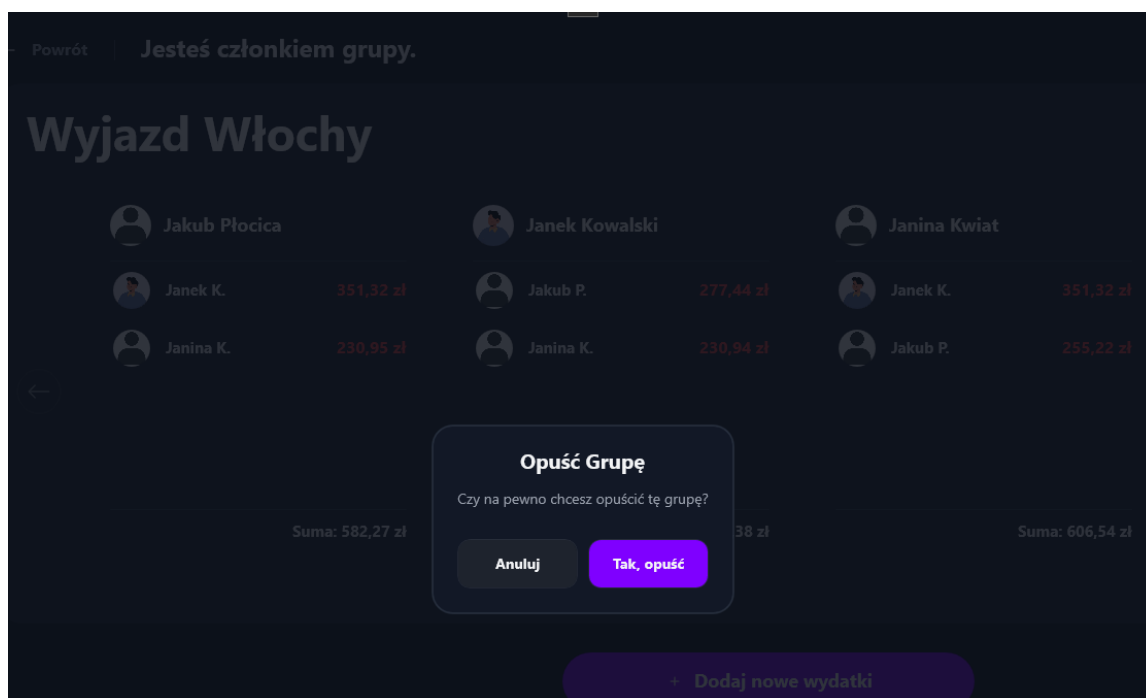
Rysunek 28: Zarządzanie członkostwem. Ręczne wpisywanie identyfikatora, notyfikacja o nowym zaproszeniu na urządzeniu docelowym oraz ścisła obsługa błędu nieistniejącego użytkownika.



Rysunek 29: Panel administracyjny. Kontrola nad uprawnieniami przypisanymi do członków (RBAC) oraz dostęp do płynnej edycji metadanych grupy (np. nazwy).



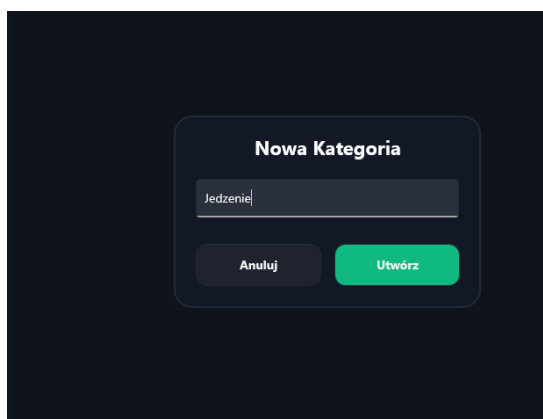
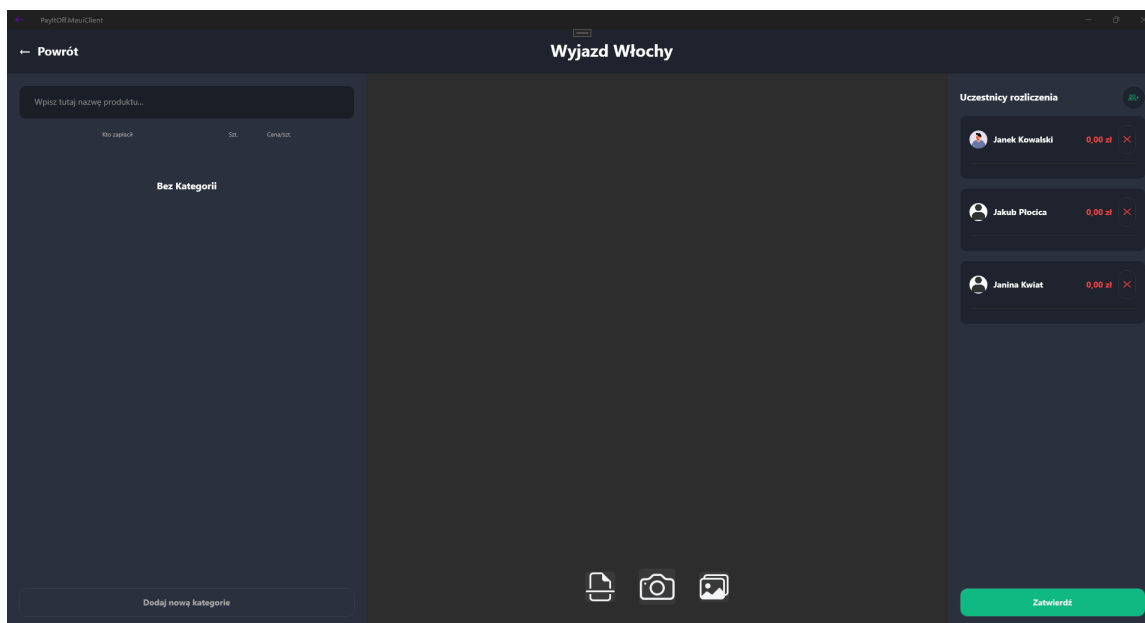
Rysunek 30: Zarządzanie uprawnieniami. Przekazanie roli administratora innemu użytkownikowi, odebranie powiadomienia w czasie rzeczywistym i ostateczny widok odświeżonego statusu.



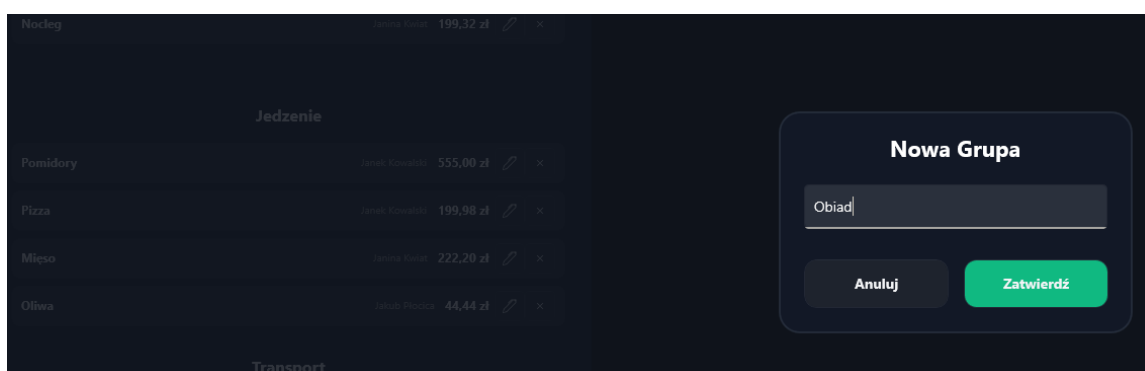
Rysunek 31: Ostatnie fazy istnienia użytkownika lub grupy. Proces bezpiecznego opuszczenia (wymaga spłaty długów) oraz panel trwałego usuwania całego środowiska.

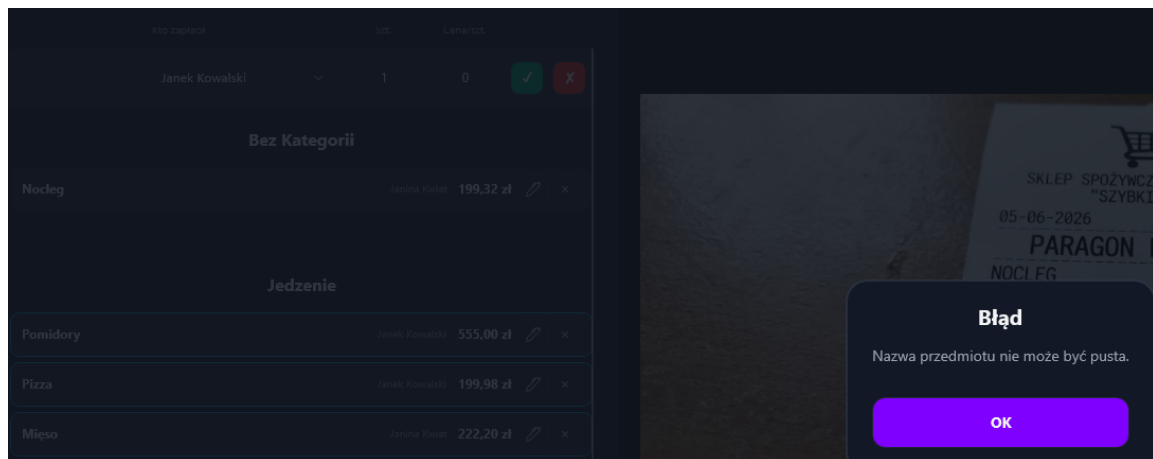
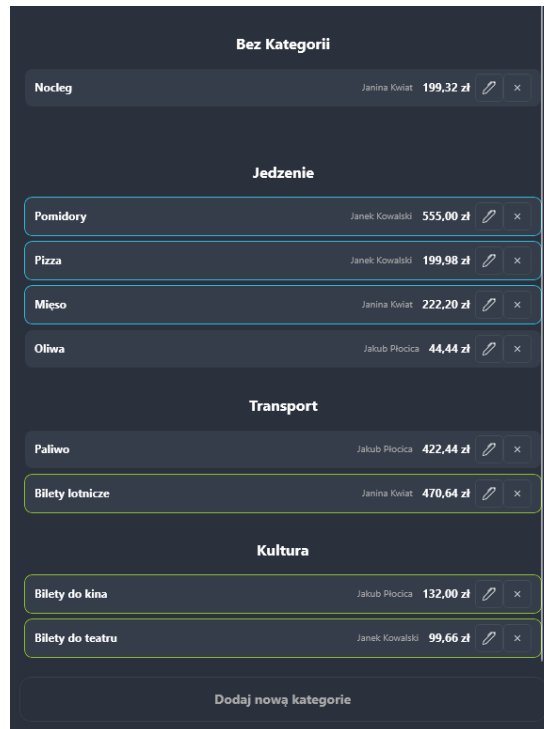
5.5. Krok 5: Księgowanie Rachunków (Wydatki)

Kluczowy etap, gdzie na podstawie zrobionych zakupów dodajemy konkretne paragony. System pozwala kategoryzować wydatki, używać parsera matematycznego w locie i dynamicznie ustalać udziały procentowe/ułamkowe z zabezpieczeniem przed błędem reszty (Penny-Drop problem).



Rysunek 32: Stan początkowy zakładki wydatków w nowej grupie oraz kreator własnych kategorii wydatkowych pomagający w analityce i utrzymaniu porządku.





Rysunek 33: Grupowanie pojedynczych produktów paragonu. Przypisywanie tagów oraz restrykcyjna walidacja po stronie klienta (zapobieganie pustym nazwom pozycji).

← Powrót

Wpisz tutaj nazwę produktu...

	Kto zapłacić		Szt.	Cena/szt.		
Pomidory	Janek Kowalski	▼	10	10*5.55	✓	✕
Paliwo	Jakub Plocica	▼	2	211.22	✓	✕
Nocleg	Janina Kwiat	▼	3	66.44	✓	✕
Bilety do kina	Jakub Plocica	▼	3	44	✓	✕
Bilety do teatru	Janek Kowalski	▼	3	31+2.22	✓	✕
Bilety lotnicze	Janina Kwiat	▼	3	156.88	✓	✕
Oliwa	Jakub Plocica	▼	2	22.22	✓	✕
Mięso	Janina Kwiat	▼	5	2*22.22	✓	✕
Pizza	Janek Kowalski	▼	6	33.33	✓	✕

Bez Kategorii

Dodaj nową kategorię

Bez Kategorii			
Pomidory	Janek Kowalski	555,00 zł	✕
Paliwo	Jakub Plocica	422,44 zł	✕
Nocleg	Janina Kwiat	199,32 zł	✕
Bilety do kina	Jakub Plocica	132,00 zł	✕
Bilety do teatru	Janek Kowalski	99,66 zł	✕
Bilety lotnicze	Janina Kwiat	470,64 zł	✕
Oliwa	Jakub Plocica	44,44 zł	✕
Mięso	Janina Kwiat	222,20 zł	✕
Pizza	Janek Kowalski	199,98 zł	✕

Rysunek 34: Innowacyjne użycie parsera matematycznego (np. wpisanie 10+5 w polu ceny) i elastyczne przydzielanie dłużników do konkretnych pozycji na rachunku.

Uczestnicy rozliczenia

Janek Kowalski

895,47 zł

X

Bilety lotnicze

Janina Kwiat

235,32 zł

X

Bilety do kina

Jakub Płocica

66,00 zł

X

Bilety do teatru

Janek Kowalski

49,83 zł

X

Paliwo

Jakub Płocica

422,44 zł

X

Oliwa

Jakub Płocica

22,22 zł

X

Nocleg

Janina Kwiat

99,66 zł

X

Jakub Płocica

473,03 zł

X

Bilety lotnicze

Janina Kwiat

235,32 zł

X

Bilety do kina

Jakub Płocica

66,00 zł

X

Bilety do teatru

Janek Kowalski

49,83 zł

X

Oliwa

Jakub Płocica

22,22 zł

X

Nocleg

Janina Kwiat

99,66 zł

X

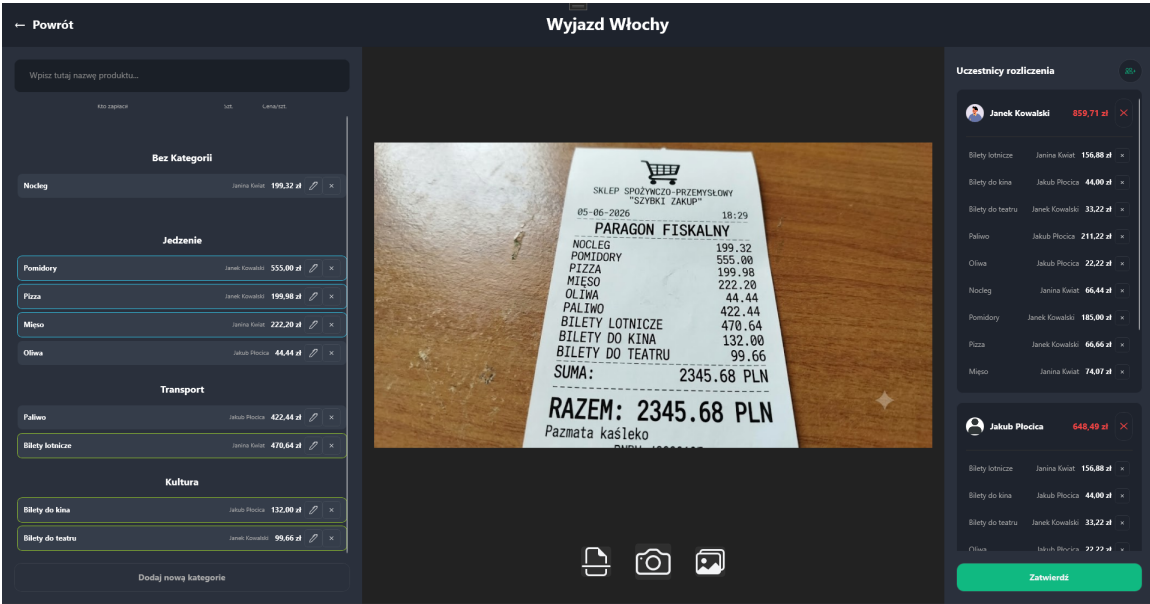
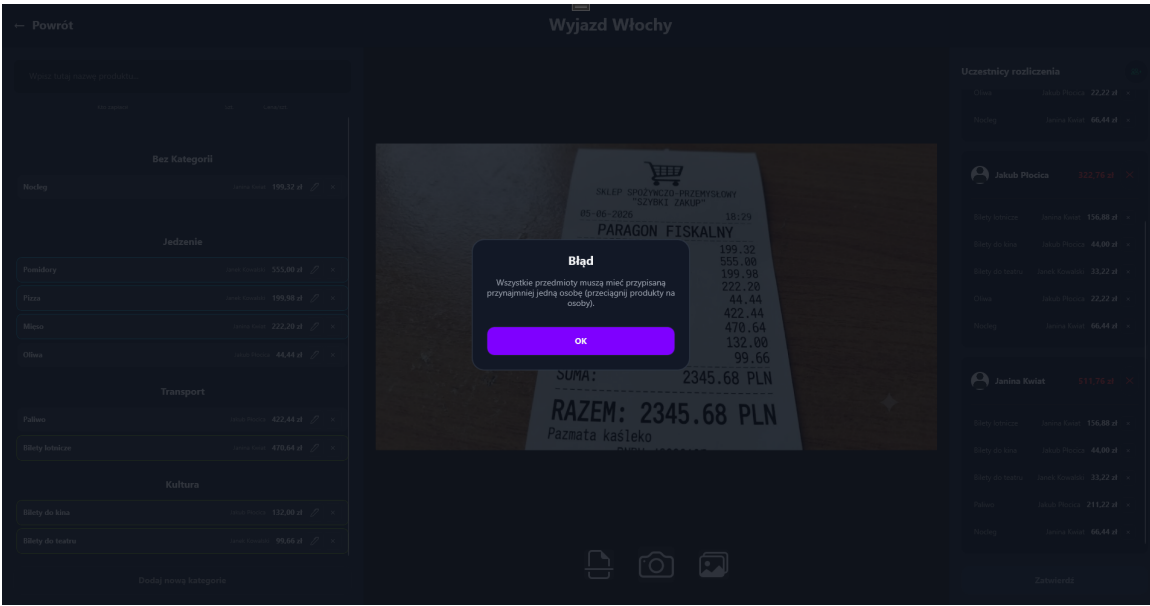
Zatwierdź

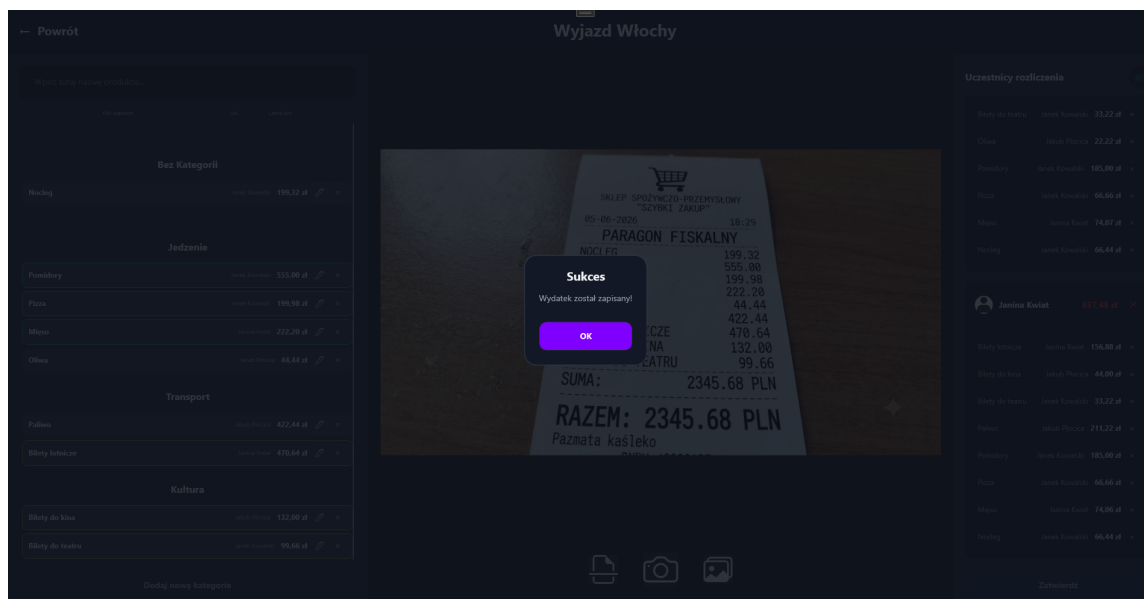
Przywróć Użytkowników

Janina Kwiat

Dodaj

Zamknij

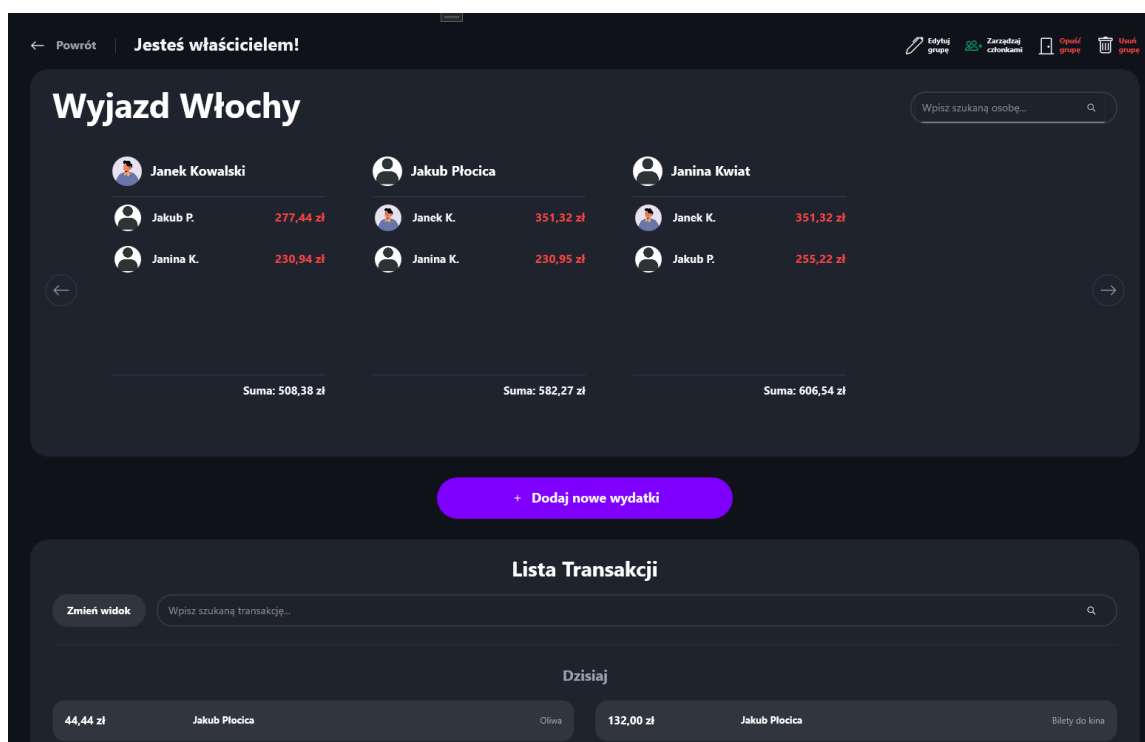




Rysunek 36: Rozwiązanie problemu reszty (Penny-Drop). Blokada zapisu przy ułamkowym nieprzypisaniu kwoty, finalne podsumowanie (Summary) i potwierdzenie sukcesu operacji serwerowej.

5.6. Krok 6: Inspekcja i Monitorowanie

Zakładka przeglądu historii grupy. Użytkownicy w każdej chwili mogą cofnąć się w czasie, odnaleźć archiwalny wydatek i zajrzeć w szczegółowy Drill-Down podziału kosztów.



Lista Transakcji

Zmień widok

Wpisz szukaną transakcję...

Q

Dzisiaj

44,44 zł

Jakub Płocica

Oliwa

132,00 zł

Jakub Płocica

Bilety do kina

422,44 zł

Jakub Płocica

Paliwo

222,20 zł

Janina Kwiat

Mięso

470,64 zł

Janina Kwiat

Bilety lotnicze

199,32 zł

Janek Kowalski

Nocleg

555,00 zł

Janek Kowalski

Pomidory

199,98 zł

Janek Kowalski

Pizza

99,66 zł

Janek Kowalski

Bilety do teatru

Rysunek 37: Inspekcja finansowa wewnątrz grupy. Widoczne podsumowanie zbalansowanego salda (kto jest winien, kto ma nadpłatę) oraz historia w ujęciu chronologicznym.

Suma: 508,38 zł

Suma: 582,27 zł

Suma: 606,54 zł

Szczegóły transakcji

Nazwa

Bilety lotnicze

Kategoria

Transport

Data zakupu

05.06.2026

Łączna kwota

470,64 zł

Oplacił(a)

Janina Kwiat

501300400

Podział kosztów

Jakub Płocica

156,88 zł

Janina Kwiat

156,88 zł

Janek Kowalski

156,88 zł

Edytuj

Zamknij

Zmień widok

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Wzrost

Rysunek 38: Szczegóły historycznego wydatku. Analiza Drill-Down umożliwiająca szybkie zweryfikowanie partycypacji kosztowej w archiwalnych rachunkach.

Szczegóły transakcji

Edytuj Wydatek

Nazwa: Przedmiot3 Kategoria: Bez Kategorii

Łączna kwota wydatku: 22,00

Podział **Pozostało: 0,00 zł**

	Janina Kwiat	11,00
	Janek Kowalski	11,00

Anuluj Zapisz

Zamknij

Szczegóły transakcji

Edytuj Wydatek

Nazwa: Przedmiot3 Kategoria: Przedmiotowo

Łączna kwota wydatku: 33,00

Podział **Pozostało: 11,00 zł**

	Janina Kwiat	11,00
	Janek Kowalski	11,00

Anuluj Zapisz

Zamknij

Szczegóły transakcji

Edytuj Wydatek

Nazwa: Przedmiot3 Kategoria: Przedmiotowo

Łączna kwota wydatku: 33,00

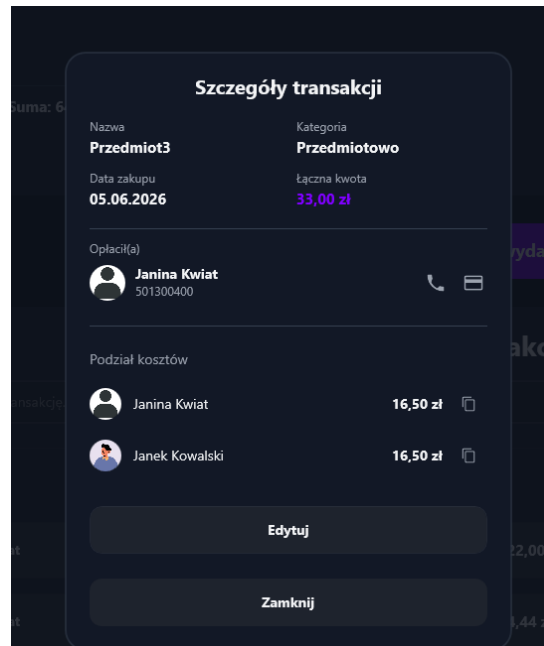
Podział **Pozostało: 0,00 zł**

	Janina Kwiat	16,50
	Janek Kowalski	16,50

Anuluj Zapisz

Zamknij

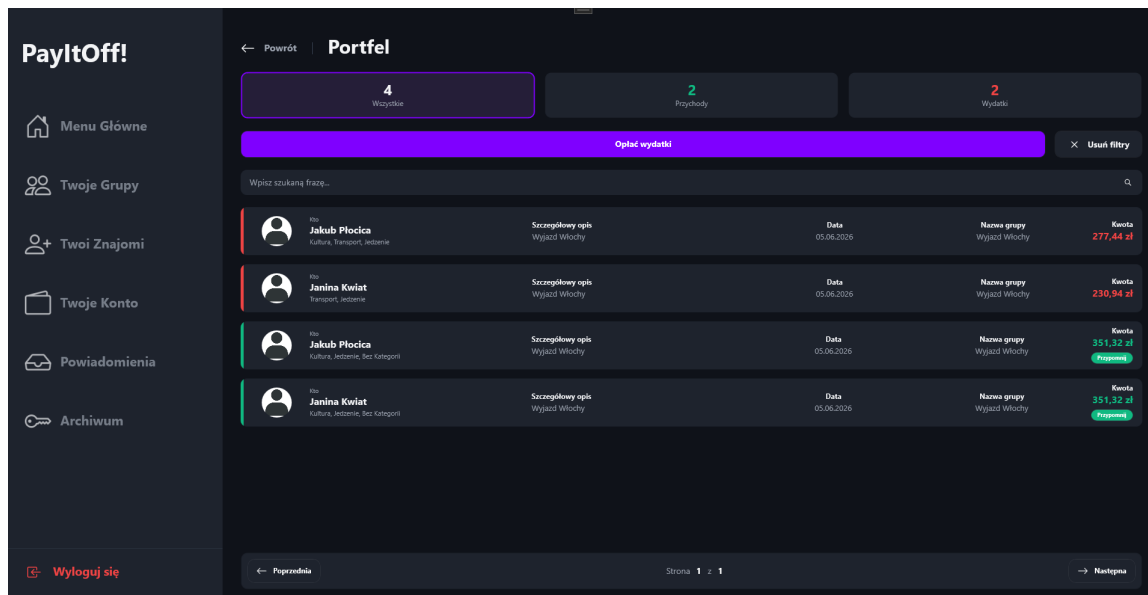
Rysunek 39: Edycja historycznego wydatku. Formularz pozwalający na modyfikację istniejących paragonów, blokada zapisu w przypadku niezbilansowania kwot oraz poprawnie rozdzielona ostateczna kwota.

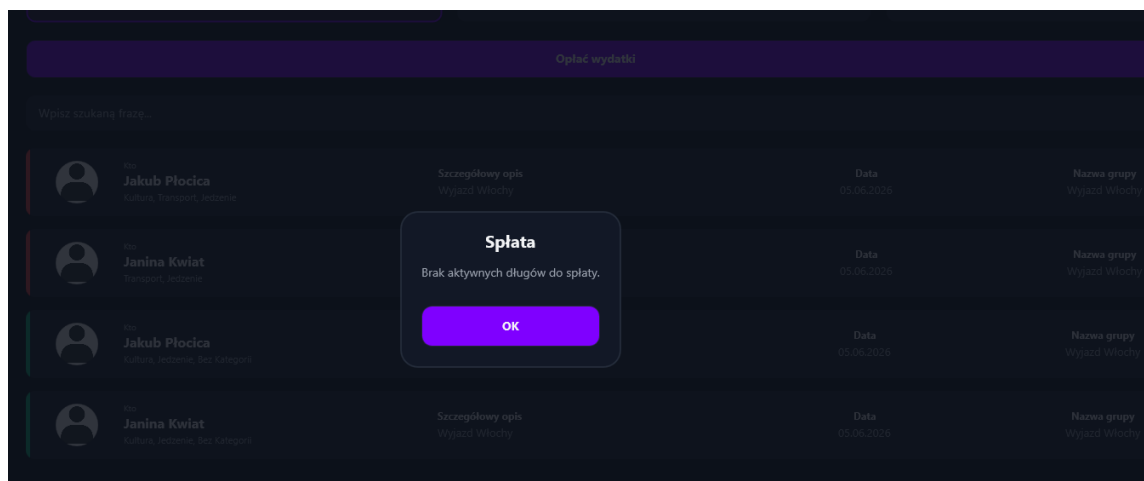


Rysunek 40: Efekt końcowy procedury edycji. Przeliczenie i aktualizacja archiwalnego podsumowania wraz z naniesionymi nowymi kwotami zadłużeń.

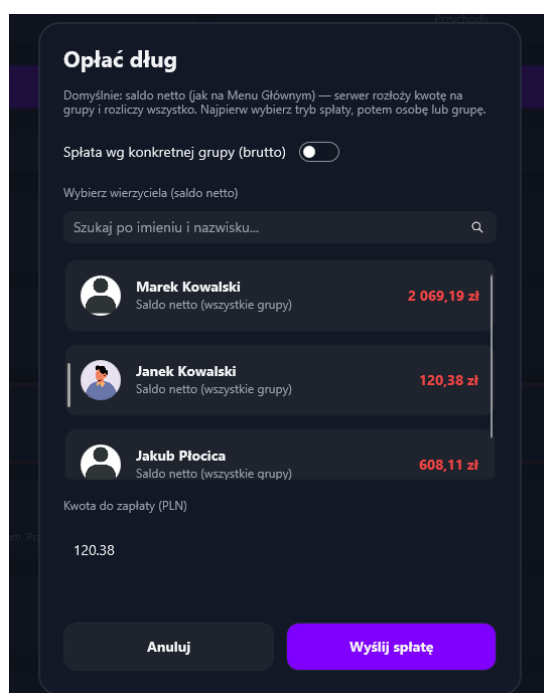
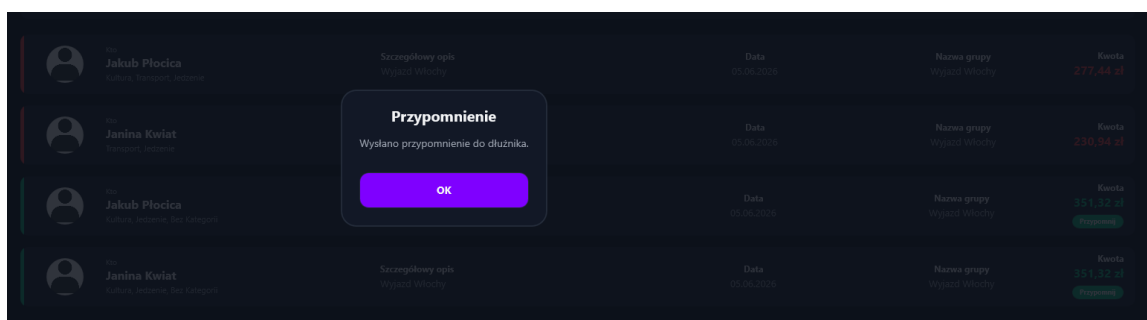
5.7. Krok 7: Zamykanie Cyklu (Rozliczanie Długów)

Ostatecznym krokiem operacji finansowej jest sprawdzenie salda i oznaczenie zwrotu pożyczki poprzez interfejs płatności. System ułatwia wysyłanie ponagleń, kompensuje długi do wartości Netto i czuwa nad uczciwością weryfikacji po stronie wierzyciela.

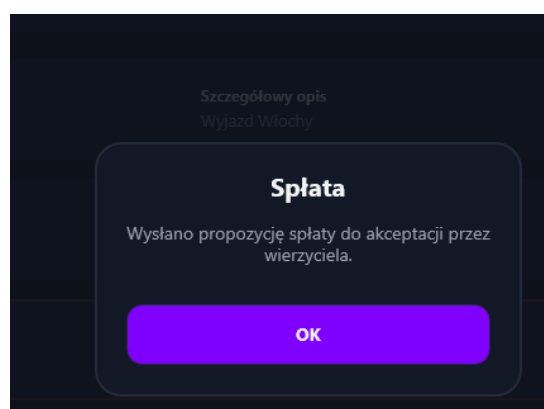




Rysunek 41: Moduł Settlement. Wszechstronny panel rozliczeniowy wyposażony w potężne filtry wektorów długu oraz miła dla oka grafika "Czystej Karty" przy wyzerowanych zobowiązaniach.



Rysunek 42: Narzędzia windykacyjne. Wysłanie przypomnienia ("Nudge") do znajomego, powiadomienie Push, oraz wywołanie inteligentnego algorytmu spłaty netto minimalizującego przepływy kapitału.



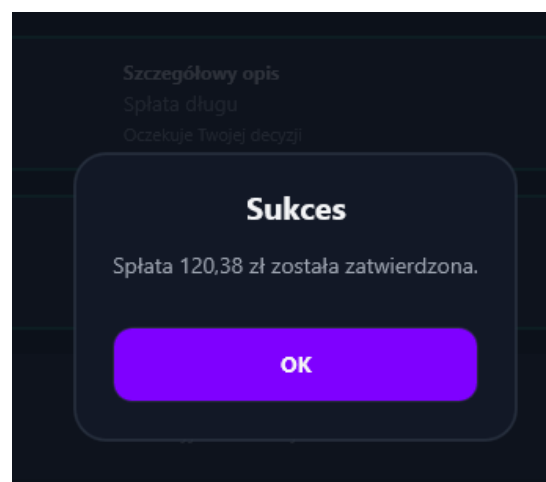
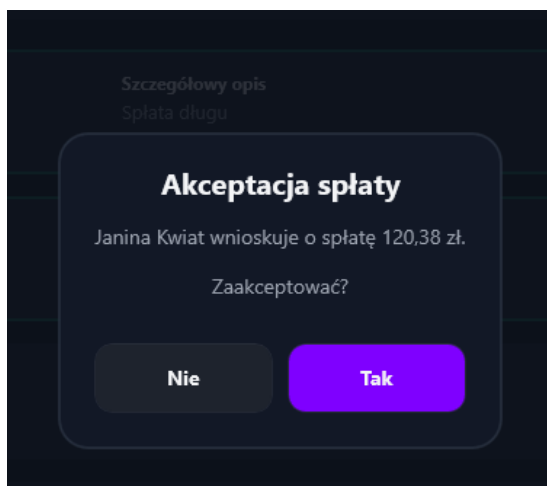
Kto	Szczegółowy opis	Data	Tytuł	Kwota
 Kto: Janek Kowalski Transfer	Szczegółowy opis Spłata długu Czekaj na akceptację wierzyciela	05.06.2026	Spłata długu	120,38 zł

Rysunek 43: Inicjacja przelewu kompensującego. Przebieg rozliczenia klasycznego (brutto), ekran potwierdzenia systemowego i zmiana statusu należności na oczekującą (Pending).

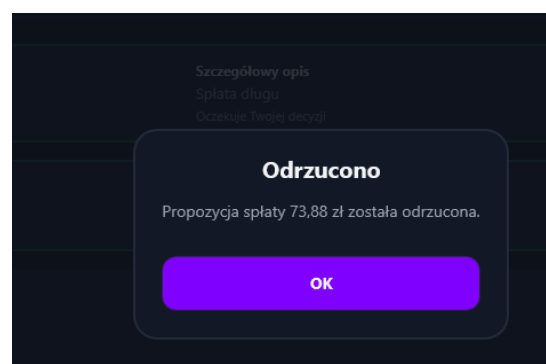
Wszystkie 5	Nieprzeczytane 5	Wymaga akcji 2	Oznacz wszystkie jako przeczytane	Usuń wszystkie powiadomienia
Jakub Płocica zadeklarował zbiorczą spłatę netto: 73,88 zł. Zaakceptuj, aby rozliczyć wszystkie powiązane grupy.	Teraz	✓	✗	🗑️
Janina Kwiat zadeklarował zbiorczą spłatę netto: 120,38 zł. Zaakceptuj, aby rozliczyć wszystkie powiązane grupy.	1 minuta temu	✓	✗	🗑️

Kto	Szczegółowy opis	Data	Tytuł	Kwota
 Kto: Jakub Płocica Transfer	Szczegółowy opis Spłata długu Odrzucona	05.06.2026	Spłata długu	73,88 zł
 Kto: Janina Kwiat Transfer	Szczegółowy opis Spłata długu Potwierdzona	05.06.2026	Spłata długu	120,38 zł

Rysunek 44: Perspektywa Wierzyciela. Otrzymanie powiadomienia wymuszającego podjęcie akcji oraz odfiltrowanie żądań wymagających zweryfikowania zaksięgowania na koncie w banku.



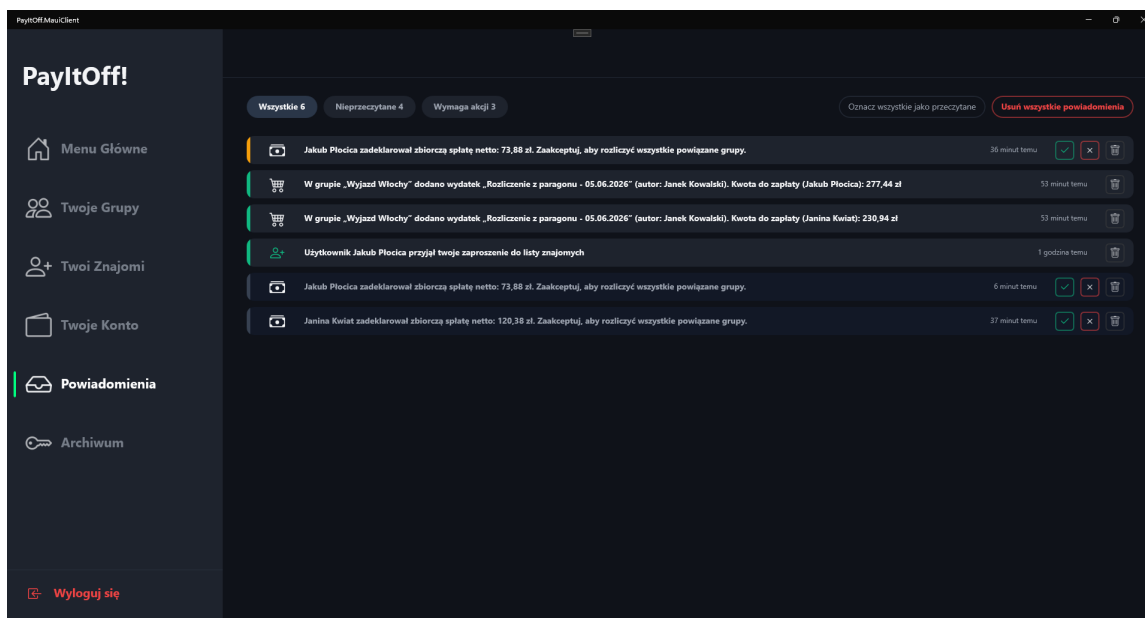
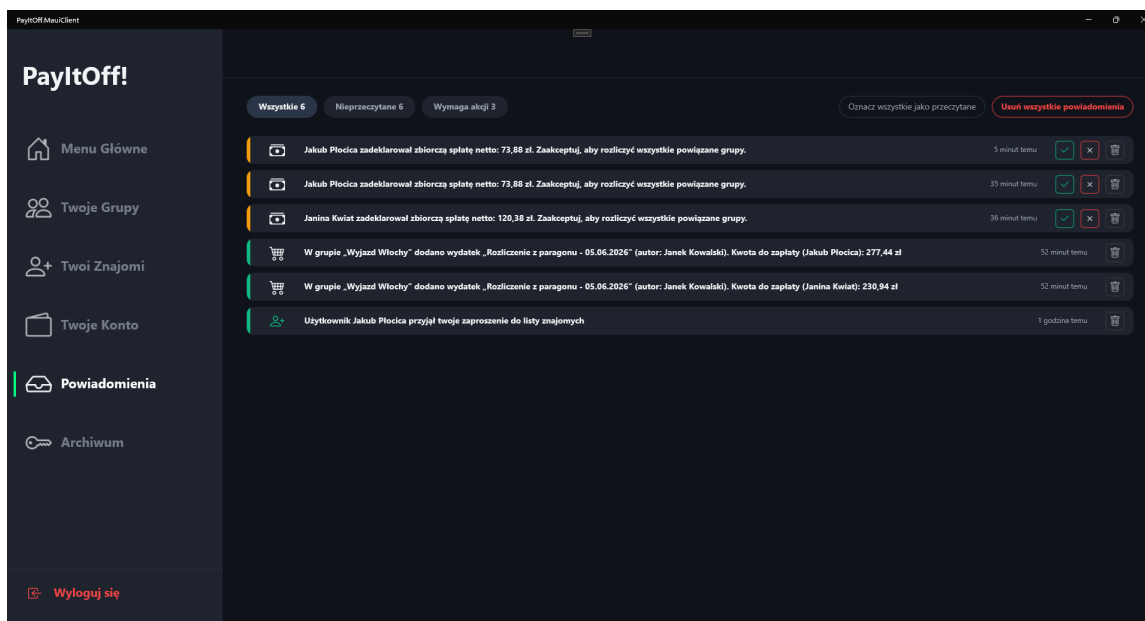
Rysunek 45: Zamknięcie pomyślnego cyklu życia długu. Wierzyciel zatwierdza odbiór płatności, aplikacja redukuje graf zadłużenia i posyła potwierdzenie zwrotne (notyfikację) do rzetelnego dłużnika.

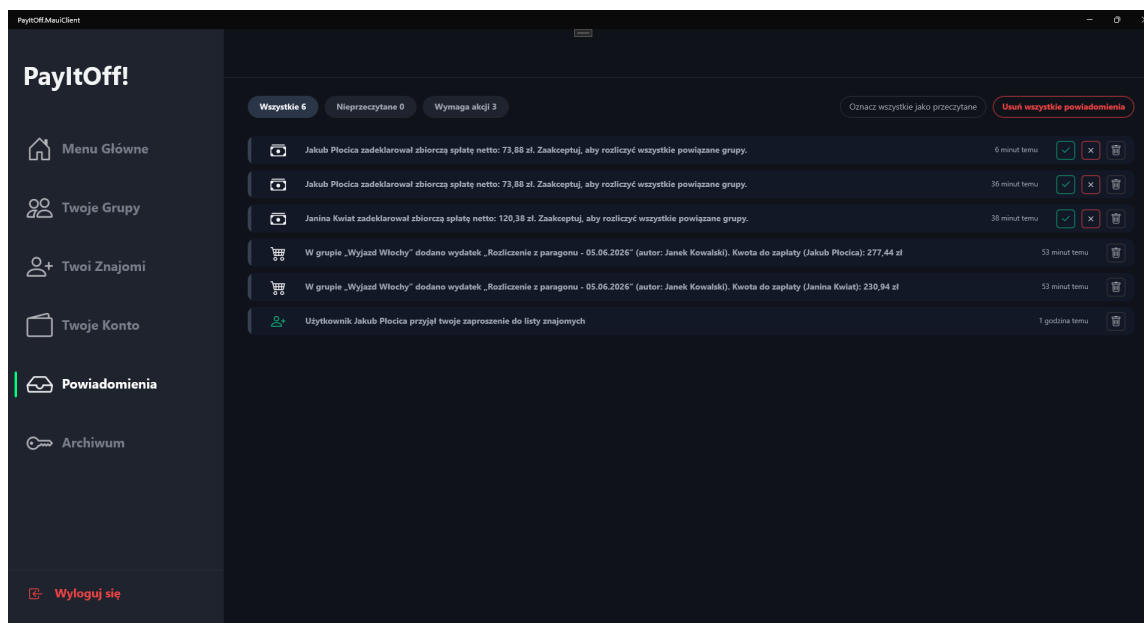


Rysunek 46: Odrzucenie fałszywego zwrotu środków (ochrona Fake-Pay). Odrzucenie deklaracji, odtworzenie wyjściowego długu i zasygnalizowanie problemu nierzetelnemu płatnikowi komunikatem błędu.

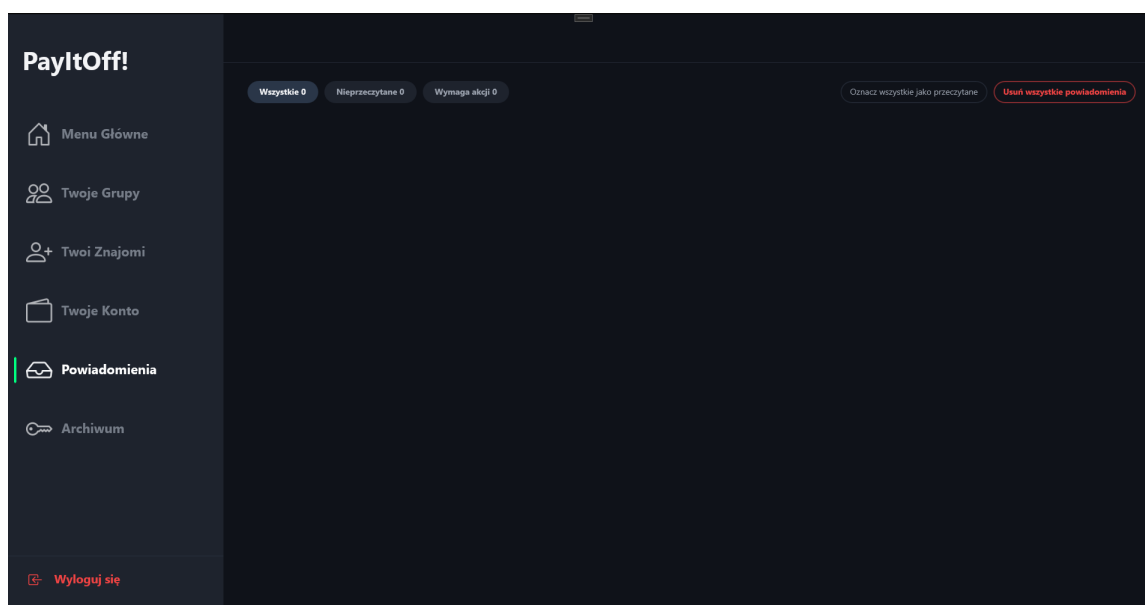
5.8. Krok 8: Centrum Powiadomień

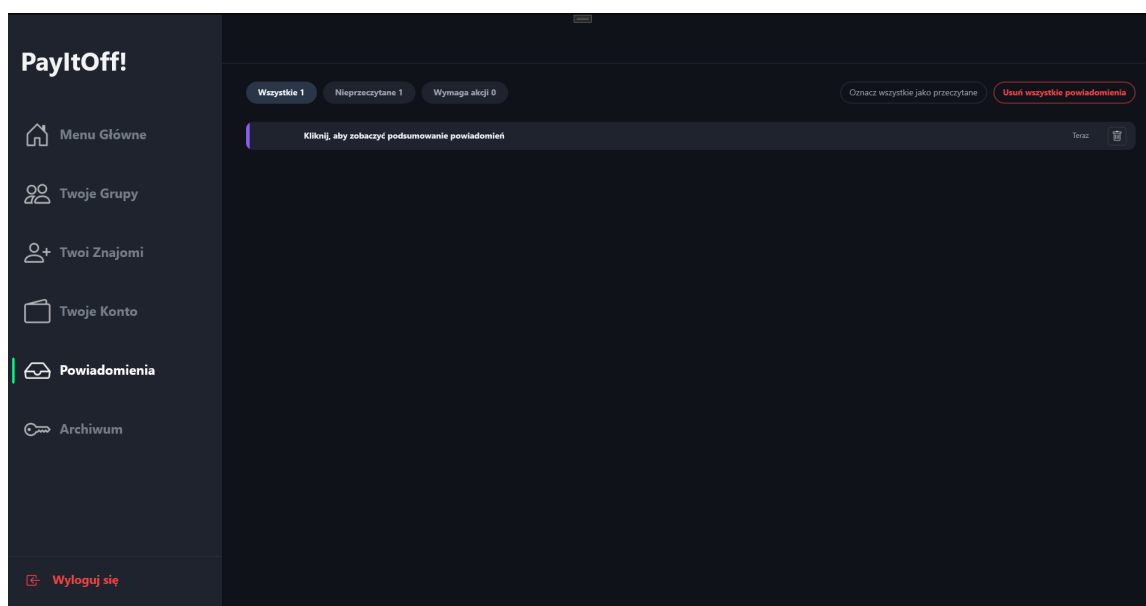
Aplikacja wyposażona jest w globalne centrum powiadomień, które agreguje zdarzenia takie jak zaproszenia, przypomnienia o długach czy modyfikacje wydatków. System wspiera masowe oznaczanie powiadomień jako przeczytane oraz ich grupowe usuwanie.





Rysunek 47: Główny widok notyfikacji. Obsługa stanów przeczytanych/nieprzeczytanych oraz szybkie oznaczanie wszystkich powiadomień.

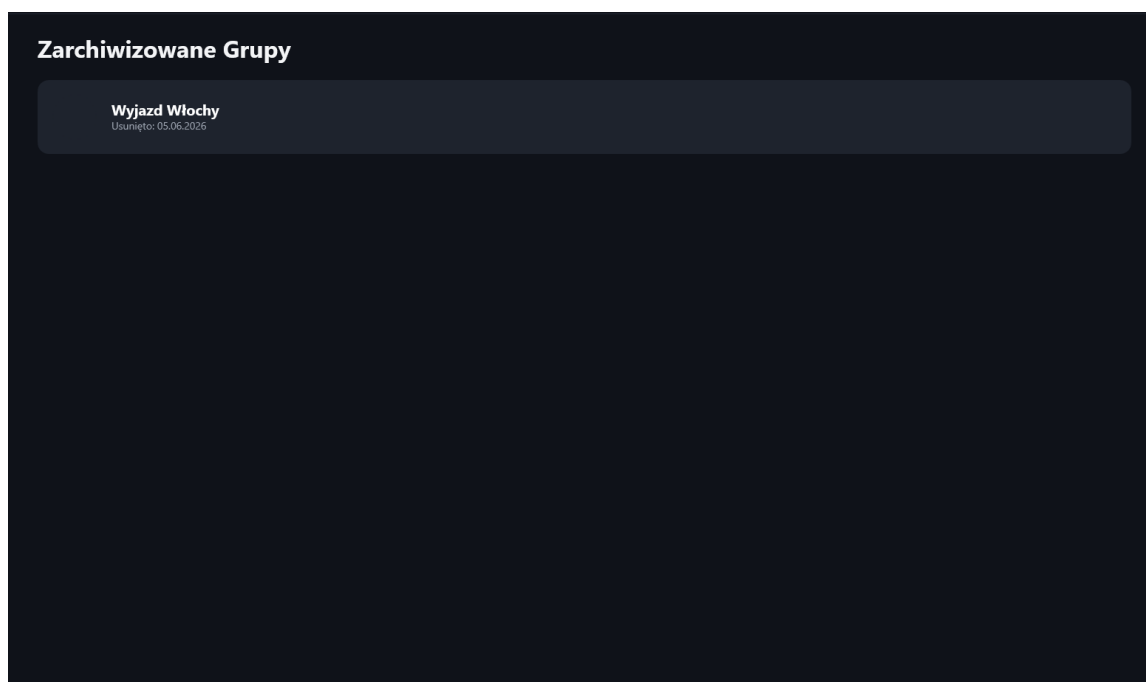
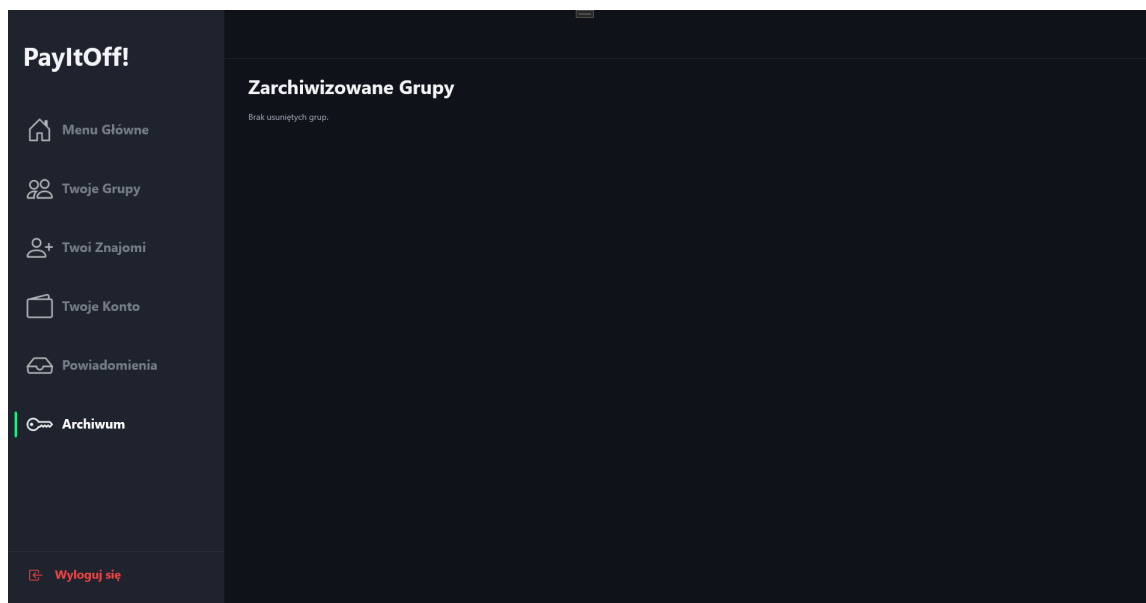


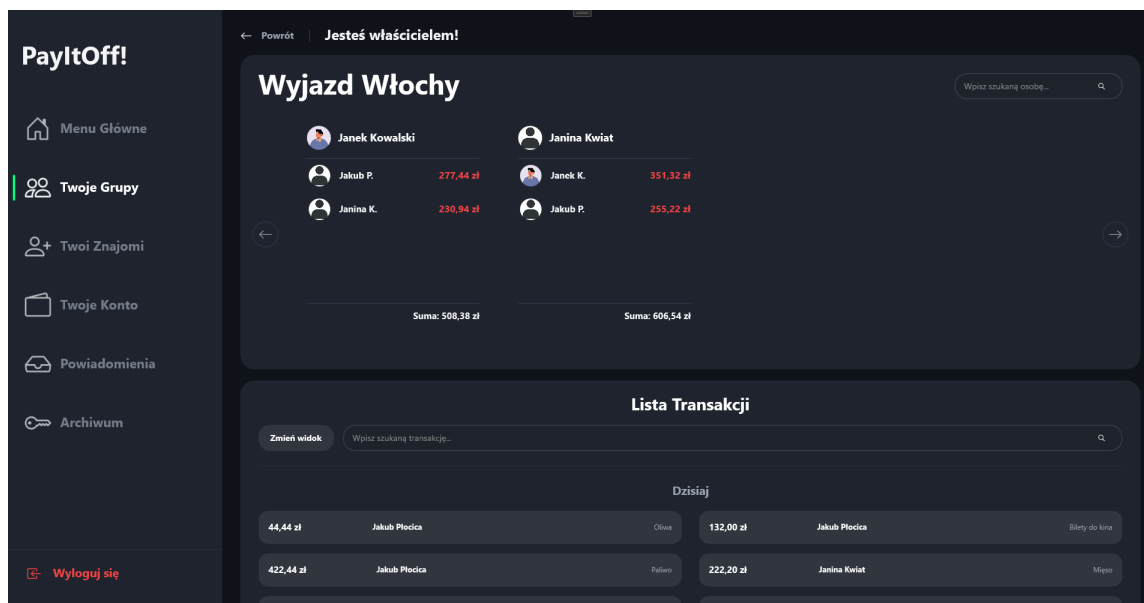


Rysunek 48: Funkcja czyszczenia skrzynki oraz obsługa inteligentnych powiadomień zbiorczych generowanych codziennie przez proces w tle (Hangfire).

5.9. Krok 9: Archiwizacja i Retencja Danych

Ostatni etap cyklu życia to bezpieczne zamykanie nieaktywnych grup (Soft-Delete) lub całkowite usunięcie danych. Zgodnie z wdrożonym wzorcem miękkiego usuwania na kolumnie DeletedAt, pozbycie się grup czy historycznych wydatków chroni relacyjne drzewo zależności w bazie. Zarchiwizowane pozycje trafiają do modułu "Archiwum", skąd mogą być tylko przeglądane w trybie odczytu.





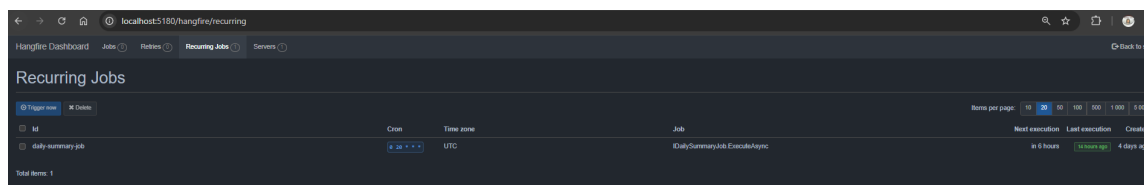
Rysunek 49: Zarządzanie retencją danych. Widok archiwum (pusty stan), wykaz usuniętych środowisk oraz odseparowany tryb "Tylko do odczytu" dla zarchiwizowanych grup wydatkowych.

6. Harmonogramowanie Zadań w Tle (Hangfire)

Aby odciążać główny wątek serwera API i zapewnić niezawodną realizację długotrwałych operacji, w projekcie wdrożono sprawdzoną bibliotekę **Hangfire**.

Hangfire pozwala na bezpieczne kolejkowanie zadań asynchronicznych typu *Fire-and-forget* oraz konfigurowanie zadań cyklicznych (*Recurring jobs*). W systemie PayItOff biblioteka ta jest podłączona do docelowej bazy danych PostgreSQL, w której zapisuje swój wewnętrzny stan oraz listę operacji do wykonania. Dzięki wbudowanemu interfejsowi graficznemu *Hangfire Dashboard* pod adresem /hangfire, deweloper może w czasie rzeczywistym podglądać aktywne procesy w tle, diagnozować błędy oraz ręcznie ponawiać przerwane zadania bez obaw o utratę danych.

Praktyczne użycie w systemie - DailySummaryJob: Aplikacja PayItOff świadomie unika "spamowania" użytkownika powiadomieniami o każdym drobnym rachunku wprowadzanym przez znajomych w ciągu dnia. W tym celu mniejsze notyfikacje są początkowo oznaczane jako "ukryte". System Hangfire posiada zdefiniowane zadanie cykliczne skonfigurowane w klasie `Program.cs` za pomocą wyrażenia Cron: `"0 20 * * *"`. Oznacza to, że każdego dnia o godzinie 20:00 proces w tle wybudza klasę `DailySummaryJob`. Serwis ten pobiera wszystkie ukryte mikro-powiadomienia przypisane do danego użytkownika z ostatnich 24 godzin, usuwa je, agreguje ich treść i tworzy jedno potężne, zbiorcze powiadomienie typu *DailySummary* ("Podsumowanie z dzisiaj"). Mechanizm ten działa w pełni asynchronicznie poza głównym wątkiem API, gwarantując wydajność aplikacji i czystość w skrzynce odbiorczej klienta.



Rysunek 50: Panel administracyjny biblioteki Hangfire. Interaktywny widok Dashboardu umożliwiający monitorowanie, wznawianie i zarządzanie kolejką zadań asynchronicznych w czasie rzeczywistym z poziomu przeglądarki.

7. Opis Wybranych Fragmentów Kodu Źródłowego

7.1. Algorytm dzielenia rachunku (Penny-Drop Problem)

Poniższy kod znajduje się w klasie `DebtCalculator`. Odpowiada on za równe dzielenie kwoty między uczestników grupy. Głównym wyzwaniem było rozwiązanie tzw. problemu zgubionego grosza, czyli sytuacji, w której kwota nie dzieli się bez reszty (np. 10 zł na 3 osoby). Kod oblicza bazową część ułamkową i rozdziela resztę w postaci pojedynczych groszy, tak by każdy rachunek sumował się co do grosza.

```
decimal baseAmount = Math.Floor((totalAmount / participantsCount) * 100) / 100;
decimal allocatedTotal = baseAmount * participantsCount;
decimal remainder = totalAmount - allocatedTotal;

int penniesToDistribute = (int)Math.Round(remainder * 100);
var sortedParticipants = participantIds.OrderBy(id => id).ToList();

for (int i = 0; i < participantsCount; i++)
{
    decimal amountForPerson = baseAmount;
    if (penniesToDistribute > 0)
    {
        amountForPerson += 0.01m;
        penniesToDistribute--;
    }
    // ... przypisanie ostatecznej wartości ułamkowej do osoby
}
```

Rysunek 51: Algorytm rozwiązywania problemu zgubionego grosza.

7.2. Logowanie i zapisywanie tokenu JWT

W aplikacji klienckiej (.NET MAUI) za komunikację z serwerem i logowanie odpowiada serwis `AuthService`. Po udanym zapytaniu HTTP do API, serwer weryfikuje hasło i zwraca bezpieczny token JWT. Ten token jest przechowywany w bezpiecznym magazynie kluczy (`SecureStorage`), by uchronić go przed kradzieżą.

```
public async Task<bool> LoginAsync(LoginRequest request)
{
    var response = await _httpClient.PostAsJsonAsync($"User/login", request);
```

```

if (response.IsSuccessStatusCode)
{
    var result = await response.Content.ReadFromJsonAsync<LoginResponse>();
    await SecureStorage.Default.SetAsync("jwt_token", result!.Token);
    return true;
}
else
{
    var errorContent = await response.Content.ReadAsStringAsync();
    // wyłapanie błędu i poinformowanie użytkownika...
}
}

```

Rysunek 52: Odbieranie i zabezpieczanie tokenu autoryzacyjnego.

7.3. Globalna Obsługa Błędów i Walidacja potoku HTTP

Aby zachować maksymalną czystość kodu w kontrolerach i zapobiec wyciekowi niespodziewanych wyjątków na zewnątrz struktury API, zastosowano niestandardowe oprogramowanie pośredniczące (*ExceptionMiddleware*). Przechwytuje ono każdy błąd na poziomie całego potoku, formatując go w ujednoliconą odpowiedź JSON. Kod połączono z architekturą **FluentValidation**, która automatycznie ocenia reguły biznesowe zanim żądanie w ogóle trafi do serwisów aplikacyjnych.

```

public async Task InvokeAsync(HttpContext httpContext)
{
    try
    {
        await _next(httpContext);
    }
    catch (Exception ex)
    {
        await HandleExceptionAsync(httpContext, ex);
    }
}

// (...) Fragment metody HandleExceptionAsync:
if (ex is ValidationException validationEx)
{
    var errors = validationEx.Errors.Select(e => e.ErrorMessage);
    await response.WriteAsJsonAsync(new {

```

```

        message = "Validation Failed",
        errors = errors
    });
}

```

Rysunek 53: Globalny system przechwytywania i ujednolicania błędów (ExceptionMiddleware).

7.4. Odpytywanie bazy przy użyciu Entity Framework

Do obsługi zapytań bazodanowych w projekcie zastosowano wzorzec Repozytorium. Poniższy krótki fragment pochodzi z `GroupDebtRepository` i pokazuje użycie LINQ oraz zapytań asynchronicznych do bazy SQL. Sprawdza on z użyciem metody `AnyAsync`, czy w wybranej grupie istnieją jeszcze jakieś nieuregulowane długi.

```

public async Task<bool> HasActiveGroupDebt(int groupId)
{
    return await _context.GroupDebts
        .Where(x => x.GroupId == groupId && x.Amount > 0)
        .AnyAsync();
}

public async Task<GroupDebt?> GetDebtAsync(int groupId, int debtorId, int creditorId)
{
    return await _context.GroupDebts
        .Where(x => x.GroupId == groupId &&
            x.DebtorId == debtorId &&
            x.CreditorId == creditorId)
        .FirstOrDefaultAsync();
}

```

Rysunek 54: Użycie Entity Framework Core do filtracji danych.

7.5. Tworzenie wydatków z użyciem Transakcji Bazy Danych

Dodawanie pojedynczego paragonu wiąże się ze stworzeniem wielu powiązanych wpisów (wydatki, pozycje na paragonie, zaktualizowanie długów osób). Zastosowano więc bezpieczne transakcje bazodanowe. Uruchamiając metodę `BeginTransactionAsync`, mamy pewność, że jeśli w trakcie obliczania nastąpi błąd, system automatycznie anuluje zmiany (Rollback), aby baza nie pozostała w stanie uszkodzonym.

```

public async Task CreateExpenseBatch(int userId, CreateExpenseBatchRequest request)
{

```

```

var group = await _groupRepository.GetGroupInfoByIdAsync(request.GroupId);
var creator = await _userRepository.GetUserByIdAsync(userId);

await _unitOfWork.BeginTransactionAsync();
try
{
    // ... tworzenie i zapisywanie wydatku Expense

    await _unitOfWork.CommitAsync(); // Wszystko ok, zapisz zmiany
}
catch (Exception ex)
{
    await _unitOfWork.RollbackAsync(); // Cofnij zmiany, zgłoś błąd
    throw;
}
}

```

Rysunek 55: Obsługa wzorca Unit Of Work w operacjach kosztowych.

7.6. Architektura MVVM i powiązanie z interfejsem

Aby kod w aplikacji mobilnej był czysty i łatwy w testowaniu, cały ekran ma oddelegowaną klasę ViewModel. Poniższy przykład pochodzi z AddExpenseViewModel. Dzięki bibliotece rozszerzeń CommunityToolkit.Mvvm i atrybutowi [ObservableProperty], system sam wie, kiedy zmieniła się nazwa grupy czy wydatek i na tej podstawie od razu odświeża napisy w plikach widoku .xaml.

```

public partial class AddExpenseViewModel : PopupViewModelBase, IQueryAttributable
{
    private readonly GroupService _groupService;

    [ObservableProperty]
    public partial int GroupId { get; set; }

    [ObservableProperty]
    public partial string GroupName { get; set; } = string.Empty;

    public ObservableCollection<ReceiptItem> BufferItems { get; } = new();

    public AddExpenseViewModel(GroupService groupService)
    {

```

```

        _groupService = groupService;
    }
}

```

Rysunek 56: Kod struktury ekranu przy użyciu wzorca MVVM.

7.7. Ewaluator Wyrażeń Matematycznych (Math Parser) w polu ceny

Aby ułatwić użytkownikom wprowadzanie skomplikowanych rachunków prosto z paragonu (np. sumowanie cen wielu małych produktów), aplikacja udostępnia funkcję automatycznego przeliczania wyrażeń matematycznych (np. $5.50 + 2 * 3$) bezpośrednio w polu tekstowym. Użyto do tego wbudowanego silnika ewaluacji danych z biblioteki `System.Data`.

```

private void ParseUnitPrice()
{
    try
    {
        if (string.IsNullOrEmpty(_unitPriceInput)) { UnitPrice = 0; return; }

        string expression = _unitPriceInput.Replace(',', '.', '.');
        var result = new System.Data.DataTable().Compute(expression, null);

        if (result != DBNull.Value && result != null)
        {
            UnitPrice = Math.Round(Convert.ToDecimal(result), 2);
        }
    }
    catch { /* Zignoruj błędne formaty podczas wpisywania */ }
}

```

Rysunek 57: Konwersja wyrażenia matematycznego z pola tekstowego w locie (Real-time Math Parser).

7.8. Reaktywny algorytm podziału kosztów na żywo (Client-Side)

Zanim rachunek trafi do ostatecznej weryfikacji na serwerze API, klient MAUI udostępnia natychmiastowy, reaktywny podgląd "kto zapłaci ile" już w momencie zaznaczania osób. Kod z modelu `ReceiptModels.cs` dokonuje "Sprawiedliwego podziału co do grosza" (rozwiązując problem ułamków) w czasie rzeczywistym i aktualizuje widok.

```

public void RecalculateShares()
{
    if (AssignedMembers.Count == 0) return;
}

```



```

decimal share = Math.Round(TotalPrice / AssignedMembers.Count, 2);
decimal totalAllocated = 0;

foreach (var m in AssignedMembers)
{
    m.OwedAmount = share;
    totalAllocated += share;
}

decimal remainder = TotalPrice - totalAllocated;
if (remainder != 0 && AssignedMembers.Count > 0)
{
    // Przypisz brakującego/nadmiarowego grosza pierwszej osobie
    var firstMember = AssignedMembers.First();
    firstMember.OwedAmount += remainder;
}
}

```

Rysunek 58: Client-Side Penny Drop – podział rachunku i rozwiązanie reszty na żywo na froncie.

7.9. Automatyczna Iniekcja Tokenu JWT w aplikacji mobilnej

Aplikacja .NET MAUI posiada zaawansowany mechanizm autoryzacji bazujący na klasie `DelegatingHandler`. Po zalogowaniu, aplikacja przechowuje token sesji w bezpiecznym magazynie `SecureStorage`. Poniższy kod przechwytuje każde żądanie HTTP wychodzące z klienta i automatycznie dokleja w nagłówku pobrany certyfikat Bearer, zapewniając bezobsługową autoryzację dla wszystkich widoków.

```

public class AuthHandler : DelegatingHandler
{
    protected override async Task<HttpResponseMessage> SendAsync(
        HttpRequestMessage request, CancellationToken cancellationToken)
    {
        var token = await SecureStorage.Default.GetAsync("jwt_token");

        if (!string.IsNullOrEmpty(token))
        {
            request.Headers.Authorization =
                new AuthenticationHeaderValue("Bearer", token);
        }
    }
}

```

```

    }

    return await base.SendAsync(request, cancellationToken);
}
}

```

Rysunek 59: Wzorzec DelegatingHandler automatyzujący dołączanie nagłówka Authorization.

7.10. Powiadomienia E-mail przy użyciu MailKit i protokołu SMTP

System powiadomień oparty jest nie tylko na alertach w aplikacji, ale również na prawdziwych wiadomościach e-mail przesyłanych do skrzynek pocztowych użytkowników. W tym celu w warstwie Aplikacji zastosowano zaawansowaną bibliotekę MailKit. Poniższy fragment serwisu zarządza nawiązaniem szyfrowanego połączenia TLS z zewnętrznym serwerem SMTP i uwierzytelnieniem nadawcy przed asynchroniczną wysyłką powiadomienia (np. o zaproszeniu do grupy).

```

public async Task SendEmailAsync(string to, string subject, string body)
{
    var email = new MimeMessage();
    email.From.Add(new MailboxAddress("PayItOff", "no-reply@payitoff.com"));
    email.To.Add(MailboxAddress.Parse(to));
    email.Subject = subject;
    email.Body = new TextPart(TextFormat.Html) { Text = body };

    using var smtp = new SmtplibClient();
    await smtp.ConnectAsync("smtp.server.com", 587, SecureSocketOptions.StartTls);
    await smtp.AuthenticateAsync("user", "password");
    await smtp.SendAsync(email);
    await smtp.DisconnectAsync(true);
}

```

Rysunek 60: Implementacja wysyłki maili z poziomu serwera C# używając standardu Mime-Message.

7.11. Dostosowanie platformy Windows (MAUI Desktop Handlers)

Aplikacja .NET MAUI działa wieloplatformowo, jednak by zachować nowoczesny wygląd na komputerach osobistych (Desktop), w klasie MauiProgram.cs zaimplementowano nie-standardowe handlersy (Mappery) pod platformę Windows. Dzięki kompilacji warunkowej #if WINDOWS, system nadpisuje domyślne zachowania formantów takich jak Entry, Picker czy Editor, usuwając natywne obramowania systemu Windows i zapobiegając niepożądanemu

przewijaniu interfejsu w górę po kliknięciu (PreventScrollOnFocus).

```
Microsoft.Maui.Handlers.EntryHandler.Mapper.AppendToMapping("NoBorder", (h, v) =>
{
#if WINDOWS
    h.PlatformView.BorderThickness = new Microsoft.UI.Xaml.Thickness(0);
    h.PlatformView.Background = new Microsoft.UI.Xaml.Media.SolidColorBrush(
        Microsoft.UI.Colors.Transparent);
    h.PlatformView.BorderBrush = new Microsoft.UI.Xaml.Media.SolidColorBrush(
        Microsoft.UI.Colors.Transparent);

    h.PlatformView.GotFocus += (s, e) =>
    {
        h.PlatformView.BorderThickness = new Microsoft.UI.Xaml.Thickness(0);
    };
#endif
});
```

Rysunek 61: Modyfikacja natywnych formantów systemu Windows w locie.

8. Instrukcja Uruchomienia Aplikacji

Aby poprawnie uruchomić system PayItOff, należy skonfigurować i uruchomić obie jego warstwy: serwerową (API wraz z bazą danych) oraz kliencką (.NET MAUI).

8.1. Wymagania wstępne

- Zainstalowane środowisko uruchomieniowe **.NET 10 SDK**.
- Zainstalowane IDE **Visual Studio 2026** z włączonymi obciążeniami roboczymi (workloads): *ASP.NET and web development* oraz *.NET Multi-platform App UI development*.
- Zainstalowane i uruchomione narzędzie **Docker Desktop** (niezbędne do konteneryzacji bazy danych i serwera API).

8.2. Krok 1: Uruchomienie Serwera i Bazy Danych (Backend)

Zarządzanie środowiskiem serwerowym oraz bazą danych odbywa się za pomocą środowisk kontenerowych, co wyklucza typowe problemy z konfiguracją środowiska lokalnego u różnych programistów.

1. Otwórz terminal (wiersz poleceń) w głównym katalogu pobranej solucji (w miejscu, gdzie znajduje się plik `docker-compose.yml`).
2. Wywołaj polecenie budujące i uruchamiające całą infrastrukturę w tle:

```
docker-compose up -d --build
```

3. Proces ten pobierze odpowiednie obrazy systemowe, uruchomi relacyjną bazę danych, a następnie zaimplementuje migracje struktury bazy (Entity Framework). Serwer Web API rozpocznie nasłuchiwanie żądań HTTP.

Alternatywa deweloperska: Serwer API można również uruchomić bezpośrednio z poziomu Visual Studio, wybierając projekt `PayItOff.Api` jako startowy (np. profil `http / https`), przy czym kontener z samą bazą danych nadal musi być aktywny.

8.3. Krok 2: Uruchomienie Klienta Desktopowego (.NET MAUI)

Gdy serwer API działa i nasłuchuje na wskazanym porcie, należy uruchomić interfejs graficzny aplikacji:

1. Otwórz główny plik solucji (`PayItOff.sln`) w środowisku Visual Studio 2026.

2. W oknie *Solution Explorer* zlokalizuj projekt kliencki o nazwie `PayIt0ff.MauiClient`.
3. Kliknij na niego prawym przyciskiem myszy i wybierz opcję **Set as Startup Project** (Ustaw jako projekt startowy).
4. Na górnym pasku narzędzi uruchamiania (w sekcji *Framework Target*) upewnij się, że wybrano maszynę docelową: **Windows Machine**.
5. Rozpocznij proces kompilacji i debugowania wciskając klawisz F5 (lub zielony przycisk Play).
6. Aplikacja skompiluje widoki XAML, uruchomi się jako natywny program okienkowy Windows i nawiąże połączenie z uruchomionym wcześniej lokalnym serwerem backendowym.

9. Repozytorium GitHub

Cały kod źródłowy aplikacji (zarówno serwer API, warstwa bazy danych, jak i sama aplikacja MAUI) znajduje się w moim prywatnym / uczelnianym repozytorium:

- **Link do projektu:** <https://github.com/Kaparee/PayItOff>

10. Podsumowanie i Wnioski

Projekt PayItOff to system do kompleksowego monitorowania i zarządzania finansami grupowymi, usprawniający proces rozliczania kosztów i długów między znajomymi.

Główne osiągnięcia projektu:

- **Interfejs użytkownika:** Stworzono intuicyjne panele mobilne i desktopowe oparte o .NET MAUI i wzorzec MVVM, zapewniające efektywne zarządzanie wydatkami.
- **Zarządzanie wydatkami:** System umożliwia tworzenie grup, dodawanie rachunków, przeliczanie długów w czasie rzeczywistym oraz odnotowywanie spłat.
- **Bezpieczeństwo danych:** Wdrożono bezpieczne logowanie oparte na tokenach JWT oraz hashowanie haseł w celu ochrony danych użytkowników.
- **Trwałość danych:** Zapewniono stabilne przechowywanie danych w relacyjnej bazie dzięki frameworkowi Entity Framework Core i wbudowanej transakcyjności.
- **Modułowa architektura:** Zastosowano warstwową architekturę (Clean Architecture) z wzorcem repozytorium i wstrzykiwaniem zależności (Dependency Injection), co poprawiło czytelność i rozszerzalność kodu.
- **Walidacja i algorytmika:** Zaimplementowano zaawansowaną walidację danych oraz matematyczną logikę podziału kwot (unikanie błędu ułamków), aby zapewnić absolutną integralność finansową.

Wyzwania i wnioski:

Realizacja projektu pozwoliła na zdobycie praktycznego doświadczenia w zakresie tworzenia projektów typu klient-serwer, wstrzykiwania zależności, zarządzania asynchronicznością i transakcjami bazodanowymi oraz projektowania interfejsów użytkownika. Tworzenie projektu znacząco poszerzyło umiejętności w budowaniu stabilnych i bezpiecznych aplikacji rozproszonych.

Potencjalny rozwój:

System może być w przyszłości rozbudowany o kilka rzeczy, głównie o integrację z prawdziwymi bramkami płatności (np. BLIK), półautomatyczne skanowanie paragonów przy użyciu modułu OCR oraz bardziej zaawansowane statystyki wydatków.